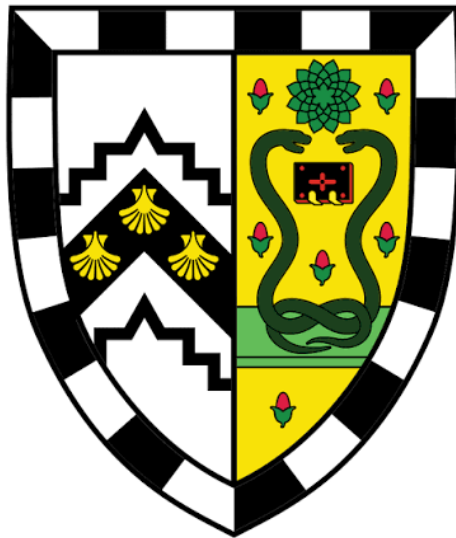


Cutting Costs: Machine Learned Cloth Simulation for Games

Computer Science Tripos - Part II



Cameron Fox
Gonville & Caius College

Year of submission: 2026

Declaration of Originality

I, the candidate for Part II of the Computer Science Tripos, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: May 2026

Signed: Cameron Fox

Proforma

Candidate:	Cameron Fox
Project Title:	Cutting Costs: Machine Learned Cloth Simulation for Games
Examination:	Computer Science Tripos Part II Dissertation - 2026
Word Count:	11,362
Code Line Count:	3,486
Project Originator:	The candidate
Day-to-Day Supervisor:	Fangcheng Zhong
Marking Supervisor:	Dr Cengiz Oztireli
Ethics Committee:	Approved Survey, #2598

Original aims

Engineer an end-to-end solution for the creation of Ubisoft's machine-learned Subspace Neural Physics models for fast cloth animations for use in video games. This includes gathering the appropriate vertex training data and implementing the training solution. Create a runtime implementation for a controllable ball-cloth model in the publicly available game engine Unity where the technique has not yet been implemented. Assess the performance of varying fidelity models against Unity's own runtime solution and evaluate the believability of the models in a user study.

Work completed

Created a solution for automatically generating banks of Alembic training animation data for the ball-cloth model in the 3D-animation software Maya. Built the required libraries for reading Alembic geometry caches in a modern Python version. Implemented the training algorithm with various data coverage techniques and an internal model saving system with an exporter to Unity. Created a single threaded, multi-threaded and GPU compute shader runtime implementation. Added the ability to render offline Alembic animations using the models from the Python environment. Conducted performance, memory and believability evaluation.

Special difficulties

None.

Contents

1	Introduction	1
1.1	Project Motivation	2
1.2	Project Overview	3
2	Preparation	3
2.1	Theoretical Background	4
2.1.1	Model Concept	4
2.1.2	Subspace Parameterisation	4
2.1.3	Model Definition	5
2.2	Technical Background	6
2.2.1	Unity	6
2.2.2	PyTorch	6
2.2.3	Maya	6
2.2.4	Alembic	6
2.3	Software Engineering	7
2.3.1	Tools	7
2.3.2	Requirements Analysis	7
2.3.3	Project Structure	8
2.4	Starting Point	8
2.4.1	Licences	8
2.4.2	Methodology	9
3	Implementation	9
3.1	Environment Setup	9
3.1.1	Library Building	9
3.1.2	Python Virtual Environment	10
3.2	Training Data Acquisition	10
3.2.1	Maya Scene Creation	10

3.2.2	Geometry Caching in Maya	11
3.2.3	Animation Automation	11
3.3	Model Training	13
3.3.1	Python Project Setup	13
3.3.2	Single Cache and Cache Bank Loading	13
3.3.3	PCA Transformation	14
3.3.4	Least-Squares Approximation	15
3.3.5	Full Initial Model Creation	15
3.3.6	Training Data Coverage Techniques	16
3.3.7	Total-Random Method	17
3.3.8	Per-Cache Methods	18
3.3.9	Loss Calculation	19
3.3.10	Hyperparameter Discovery	20
3.3.11	Model Saving and Unity Exporter	20
3.3.12	Extension: Offline Simulations and PCA Visualisation	21
3.4	Runtime Implementation	21
3.4.1	Model Loading	21
3.4.2	Solver Intervals	22
3.4.3	CPU Runtime Solver	22
3.5	Extensions	24
3.5.1	FBX Pipeline	24
3.5.2	GPU Compute Shader Implementation	24
3.5.3	Aesthetic Dampening	26
3.5.4	Multithreading with the Unity Job System	27
3.6	Evaluation Tooling	27
3.7	Repository Overview	28
4	Evaluation	28
4.1	Models and Runtime Implementations	28
4.2	Uncapped Model Performance	29

4.2.1	Single Instance	29
4.2.2	Multi Instance	30
4.3	Capped Model Performance	30
4.3.1	Single Instance	31
4.3.2	Multi Instance	31
4.4	Profiler Frame Analysis	32
4.4.1	CPU-backend/multi-threaded-deformation Frame Analysis . .	32
4.4.2	Compute Shader Frame Analysis	33
4.5	Memory	33
4.6	User Study	34
4.6.1	Survey	34
4.6.2	Results	35
4.7	Training	36
4.8	Qualitative Comparison	36
5	Conclusion	37
A	Appendix	43
A.1	Listings	43
A.2	Survey	46

Terminology

- **ML**: Machine Learning
- **NN**: Neural Network
- **PCA**: Principle Component Analysis
- **ONNX**: Open Neural Network Exchange
- **SNP**: Subspace Neural Physics [12]
- **DOF(s)**: Degree(s) of Freedom
- **abc**: Alembic Geometry Cache
- **mcx**: Maya Geometry Cache
- **HLSL**: High-Level Shader Language

1 Introduction

Video game studios often grapple with per-frame budgeting exercises; allocating time spent rendering graphics, updating AI enemies, performing player input, and any other number of game-world elements [25, 17, 11]. This resource allocation is typically platform dependent too [21, 34], where each platform has a different set of resource specifications. This highlights the need for easy-use levers [1] for developers to manage the *cost* of these individual facets whilst maximising their *quality*.

Triple-A studios are particularly challenged by this exercise as players increasingly expect “realism” in their game worlds, especially graphically [20, 5]. This quality is particularly realised in physics simulations where game-objects aim to behave comparatively to their real-world counterparts. This facet of the game world is, however, particularly expensive [1]. Running increasingly physically accurate simulations of the player’s clothes, for example, would be a luxury to afford in the per-frame budget but could be a commercial differentiator when marketing the game. This cost is especially apparent as run-time soft bodies are typically simulated on the CPU [22, 36] where other important game logic takes place.

What options do developers currently have?

Traditional soft-body simulation techniques (like mass-spring) are generalised and can be varied in computation-time and quality by reducing the number of iterations, vertex-resolution of the model and the number of interacting objects [4]. Developers could also use novel approaches to *believably* mimic these behaviours more efficiently, for example using pre-rendered animations where possible [29, 35].

What is wrong with these options?

The downside to this novel-approach method is that each may drastically differ per platform. For example, storing many animations may not be a problem on PC but it certainly could be on mobile. Therefore, `flag_wave_PC` and `flag_wave_iOS` now exist as separate entities that need to be maintained and may implement behaviourally and functionally differing algorithms.

The overheads of a general soft body solver are often not merited in heavily constrained game environments where the parameters of the simulation are known beforehand. For example, a ball kicked directly into a football net during a penalty shoot-out would exhibit a subset of all possible interactions that could take place between a ball and four cloth instances as the walls of the net; for example, the degrees of freedom (DOFs) which would allow the back of the net to cave inwards is unneeded if the ball can never collide with the net from behind. Therefore through a dimensionality reduction of the simulation [33], it may be possible to yield higher quality simulations within the same computational budget. Likewise, we could create simulations which are more performant and have the same visual quality.

How can we address these problems?

Neural networks (NNs) are a particularly useful idea in addressing both the possible gains of a constrained physics scenario and for solving the per-platform problem. Their sizes (as determined by their depth and layer size) and consequent computation times during execution are variable, yet an algorithm that uses them can abstract over this. These networks can inherently become smaller if we limit the DOFs of the simulation to only those most needed. Moreover, their quality can inherently be improved if we target the training to a certain set of interactions, which is exactly the case for many gameplay interactions.

This provides reason to believe there is potential for performance gains to be made in any constrained problem executing, like current methods, on the CPU. More importantly, NNs can be executed on the GPU unlike traditional solvers [22, 36] giving developers the power to move the problem almost entirely off the CPU.

Epic Games, Unity and Electronic Arts have all realised the potential for this NN-aided method in the specialised use case of clothing: Epic Games have implemented a machine learned (ML) clothes deformer in their Unreal Engine [25] and Unity’s *Time Ghost* and EA’s *Madden NFL 21* also both use custom NN models for clothes [32, 15].

The case for this approach in general for soft body deformation was made by Ubisoft and McGill University in their 2019 paper *Subspace Neural Physics: Fast Data-Driven Interactive Simulation*[12]. The technique described used dimensionality reduction before training a corrective NN on top of a least-squares approximation; the models were shown to be highly performant and accurate.

1.1 Project Motivation

While this technique has been identified as a catalyst for embedding many more physical simulations into a game world, there does not yet exist an end-to-end open source solution from data acquisition to game-engine implementation or an example format for transporting these models. Creating this would allow for rapid and scalable creation and deployment of new soft body simulations in games. By demonstrating deployment of a cloth model in the publicly available game engine Unity, this technique can be shown to be architecturally congruent between fidelities and as workloads shift between compute resources. Therefore, this technique could unburden developers from the maintenance of algorithmically differing per-platform solutions (with different visual styles) and provide meaningful levers for balancing the computational budget sheet.

While partial open source solutions exist for training, these do not use the specialised Alembic file format for handling training data which crucially allows for a direct pipeline from Maya physics simulations. Character skin deformations using similar NN techniques are available in some engines, but these do not use the dynamic corrective approach (as motivated by Ubisoft) which can be applied to other kinds of soft-body scenes. This project will demonstrate how the technique outlined by Ubisoft can be implemented for generic soft bodies — using a cloth-sheet example

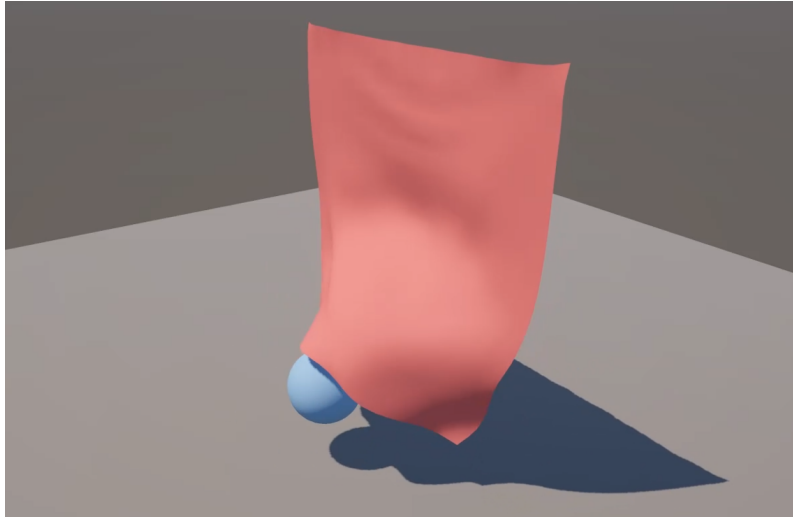


Figure 1.1: The runtime implementation of the trained cloth model within Unity as the player moves the ball from right to left.

throughout the project — in the popular game engine Unity [39].

1.2 Project Overview

In this project I have engineered an end-to-end implementation of Ubisoft’s *Subspace Neural Physics* (SNP) technique for fast cloth simulations in real-time video games. I developed the models into a transportable file structure with little overhead in editor set-up as demonstrated by their Unity implementation.

The solution includes automated training data acquisition using Maya, an implementation of the training method described by Ubisoft to obtain NN based models and, finally, solutions for using them at run time in Unity.

I have created a CPU side implementation which can interface with Unity’s NN *Inference Engine* both synchronously and asynchronously and I have implemented the models using a compute shader which keeps network output entirely on the GPU. I have also used the Unity Job System to multi-thread the mesh deformation on CPU bound runtime implementations.

I have evaluated the performance of the runtime implementations against Unity’s own CPU implementation [22] in both time and memory against the fidelity and number of instances in the scene. The performance was also evaluated against Maya’s own simulation time. I have tested the *believability* of the runtime implementations in a user study against the believability of the Maya simulations from which the models were trained using.

2 Preparation

This chapter details the technical research undertaken before implementing Ubisoft’s SNP technique, the choice of tools and the environment set-up required for training.

2.1 Theoretical Background

My research included talks, blogs and academic papers. I watched *Machine Learning, Physics Simulation, Kolmogorov Complexity, and Squishy Bunnies*[13], a 2020 GDC talk by Daniel Holden for an overview of Ubisoft and McGill University’s paper on ML physics simulation for games. I read Holden’s blog [28] on the research as well as the formal paper *Subspace Neural Physics: Fast Data-Driven Interactive Simulation*. This required me to understand Principle Component Analysis (PCA) for data compression; to learn about this, I read IBM’s article [31].

2.1.1 Model Concept

Given a physically animated object A , such as cloth, which is made of m vertices and external actors summarised as variables p , our aim is to create a model that can predict the next set of vertex positions of A based on the current and preceding positions of the vertices and the current values of the external variables. A single “frame” of A can be described as a vector m_t of length $3v$ which is simply the flattening of A ’s v vertex positions. An *animation* of A is described by contiguous frames m_t, m_{t+1}, m_{t+2} , etc... for discrete values of t ($t \in \mathbb{N}$).

In this project, A is a cloth mesh and the external variables p are a moving ball’s x, y coordinates.

2.1.2 Subspace Parameterisation

Given the physical nature of object A , the m vertices do not all move independently. It is therefore the case that there are far fewer observable DOFs that need consideration for training a model to deform A [2]. Consequently, we use PCA to extract the major modes of deformation (the DOFs used by the model) that A can undergo; this new vector space minimising the error in reconstruction of the original training set [7]. In order to use PCA, the vertex positions are centred and normalised - with these vectors saved for transforming the model outputs back to usable vertex positions later. We then perform the analysis on a representative subset of the normalised vertex frames to extract w deformation components each of length $3v$. This gives a reduced basis for describing a frame of A as a vector of length w and a matrix $\mathbf{W}$ for performing this change in basis (each row of $\mathbf{W}$ being the mode of deformation of length $3v$). This change of basis leads to an error in reproduction of the original data, however the number of bases can be adjusted to maintain this within a threshold.

We can now describe a frame of A as the vector w_t and an *animation* of A as w_t, w_{t+1}, w_{t+2} , etc. The corresponding vertex positions described by a reduced-dimension frame w_t is the linear combination of elements of w_t with their corresponding mode of deformation (row of $\mathbf{W}$); then being denormalised (using the earlier-saved standard deviation and mean of the training set’s full frames).

2.1.3 Model Definition

Using two given initial frames of A , the goal is to calculate a set of frames w_t^* as a best estimate for w_t , the ground truth of how the cloth should move. We make predictions for w_t^* using previous frames w_{t-1}^* and w_{t-2}^* , as well as the current values of external variables given as p_t . To be able to make the first predictions, we take w_0^* and w_1^* to be a known initial start state taken from the first two frames of the training data:

$$\begin{aligned} w_0^* &= w_0 \\ w_1^* &= w_1 \end{aligned}$$

To calculate w_t^* , the model first uses a least-squares approximation for the next set of vertex positions of A based solely on the immediately preceding position and velocity of the PCA coordinates. That is, we find vectors α and β such that:

$$w_t^{ls} = \alpha \cdot w_{t-1}^* + \beta \cdot (w_{t-1}^* - w_{t-2}^*)$$

This represents the physical approximation of each vertex tending towards some rest state whilst simultaneously moving based on its inertial movement. We can solve for α and β using a least-squares calculation for each component m using the matrix form

$$\begin{bmatrix} w_{m,1} & w_{m,1} - w_{m,0} \\ \vdots & \vdots \\ w_{m,t} & w_{m,t} - w_{m,t-1} \end{bmatrix} \begin{bmatrix} \alpha_m \\ \beta_m \end{bmatrix} = \begin{bmatrix} w_{m,2} \\ \vdots \\ w_{m,t+1} \end{bmatrix}$$

where $w_{m,1}$ is coordinate m of the first transformed frame of the training animation.

We then train a corrective neural network Φ which offsets w_t^{ls} based on the approximation itself, the previous frame w_{t-1}^* and the current values of the external variables.

Given approximation w_t^{ls} calculated as $w_t^{ls} = \alpha \cdot w_{t-1}^* + \beta \cdot (w_{t-1}^* - w_{t-2}^*)$, apply the neural network correction

$$w_t^* = w_t^{ls} + \Phi(w_t^{ls}, w_{t-1}^*, p_t)$$

where w_t^* is our final prediction for the true value w_t .

We train this NN by considering a *window* of frames in the training data. This window is a continuous set of frames of length l taken from the transformed training data. We use the two first frames of the window which are w_j and w_{j+1} , perturbed by some noise for w_j^* and w_{j+1}^* . These are used to calculate w_{j+2}^* . We then use this and w_{j+1}^* to predict w_{j+3}^* . We continue like this to calculate the rest of the window, reaching w_{j+l-1}^* .

We then calculate the average loss per frame over the window of absolute position and inter-frame velocity

$$\frac{1}{l-2}[(w_{j+2}^* - w_{j+2}) + \dots + (w_{j+l-1}^* - w_{j+l-1}) + ((w_{j+2}^* - w_{j+1}^*) - (w_{j+2} - w_{j+1})) + \dots]$$

This averaging technique teaches the network to favour smaller per-frame corrections that lead to less loss over time in both absolute position and motion rather than an aggressive singular correction [12]. This loss is then fed into an AmsGrad optimiser [9].

2.2 Technical Background

The foundational technical decisions for this project were the methodologies for training data collection, the libraries to use for ML training and the target engine for the runtime implementation.

2.2.1 Unity

I chose to use Unity as a target deployment engine for the runtime implementation. I have extensive experience using the engine and, more widely, it is a popular choice in the game development community and in industry [39]. Consequently, it is where the system may be most useful to others; both for those already using Unity, and for those wishing to implement the models in their own engine this would serve as a relatively high level demonstrative example project. Unity is also equipped with its own NN API, the *Inference Engine* [38] (previously *Sentis*), which has proven successful in related Unity-demo projects like *Time Ghost* [32].

Unreal Engine (UE) with its Neural Network Engine would have been the other viable alternative for its popularity in industry and as another engine I have professional experience with, however its much more esoteric API would not have made the example implementation as readable to developers without substantial UE experience — especially for a GPU implementation [26].

2.2.2 PyTorch

For network training, I chose to use PyTorch for its abundant documentation and its ability to export to the ONNX format which Unity’s *Inference Engine* can use. I also used TensorBoard for training loss visualisation and its ability to log 3D geometry.

To learn about PyTorch, I used the documentation and a series of tutorial videos [37, 23].

2.2.3 Maya

I chose to use Maya for gathering vertex data of high quality offline physics simulations; this was used in Ubisoft’s paper and would therefore provide a useful point of qualitative comparison in this project [12]. Maya also includes a scripting system which would be useful for automated data collection.

2.2.4 Alembic

After researching possible geometry cache formats, I chose to use the specialised open-source framework *Alembic* for its frame-by-frame vertex caching [8] and its

dedicated set of Python bindings. This makes it far superior to a skeletal format for static meshes like FBX. It is also exportable from Maya.

The Alembic file format, originally developed by *Lucas Film*, is intended to be an efficient store for geometry caches of 3D scene data. An Alembic archive (file) stores scene objects in an acyclic directed graph which creates a parent-child tree [8].

These objects contain structures of *Properties* which are interpreted as different *Schemas*. An object may have a *CompoundProperty* which is a special container-type property that can also identify it as a standard complex object like a *PolyMesh*. Its sub-properties could contain samples of the vertex positions at different times as *SimpleProperties* (the kind of properties that hold the actual data). By interpreting the top-level object as an *IPolyMeshSchema*, we can simply abstract over much of this underlying implementation.

In fixed-frame-rate training where we work with complete scene information (as specified in the Maya export), we assume all animated objects have the same number of samples and each of these occur at the same intervals in time.

2.3 Software Engineering

2.3.1 Tools

The project was created in a Windows environment: the standard OS in the games development industry [27, 24]. I chose to use Visual Studio Code for the Python project and Visual Studio for the runtime implementation for its good Unity C# compatibility. I backed up the project periodically on an external hard-drive and remotely on a private Git repository.

2.3.2 Requirements Analysis

The success criteria for the project (as defined in the proposal) was to recreate the runtime ball-cloth scene which was also used as a testing instance in the original Ubisoft study. This objective was laid out as follows:

- Gather vertex data from cloth simulations using an appropriate tool.
- Train the required NNs on this data such that given the current position of the moving sphere and the previous n frames of vertex-positions of the cloth we can predict the subsequent position of the cloth's vertices.
- Create a runtime implementation within Unity with a controllable sphere and instance of the cloth implementation.

Functionally, this scene should demonstrate the cloth object deforming using the SNP model in real time by allowing a user to control the ball that can interact with the cloth model. Non-functionally, the scene should be at least as performant as the same scene using Unity's own cloth model and should be as *believable* as the Maya training animations as demonstrated in a user study.

I have omitted vertex normal network training and visible intersection methods from the original SNP paper in order to bring the project into scope for this dissertation [12].

2.3.3 Project Structure

The project is structured as three components which act as a pipeline: data acquisition from Maya, the training environment and the runtime implementation in Unity. These aspects can each be coded separately and kept in their own environments with only data crossing between.

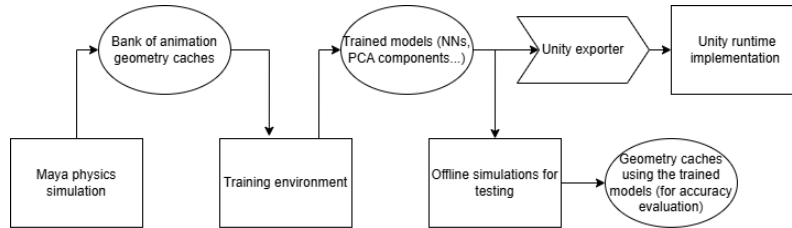


Figure 2.1: Project pipeline overview.

Maya can store user scripts internally. The Python project will be kept in its own virtual environment, and the implementation will be in its own Unity project.

2.4 Starting Point

The starting point for this project was a completely fresh Windows installation. I had researched the possible software solutions and libraries I would need to use (but had not yet built) in order for the project to be successful. I had existing knowledge and experience of Unity and general 3D-modelling software. I had foundational knowledge of ML techniques from Part-IA’s Machine Learning and Real-World Data but had never undertaken a large ML project before. I had some existing knowledge of NumPy from Part IA’s Scientific Computing.

I had never trained an NN, or used PyTorch or related tools like TensorBoard. I had also never compiled libraries with multiple dependencies before, or used Alembic or Maya.

For the compute shader extension (Section 3.5.2), I had only ever briefly written HLSL code for Part IA Graphics where most of the set-up between CPU and GPU was given as boilerplate. I had never coded a shader in Unity or used its Job System (Section 3.5.4).

2.4.1 Licences

The Unity installation used a Personal licence, as the project (or past personal projects) had not exceeded the threshold income. Maya was installed on a student licence and the project abides by the non-commercial use of its outputs as such.

The Boost Software Licence - Version 1.0 permits use of the built libraries whereby the work only uses it as an executable. Imath, Alembic, Sci-Kit, NumPy and Py-

Torch are released under the BSD-3-Clause or BSD style clauses which permit their use for research as long as the copyright notices are present in the license file of my own work. All licences are in the source code zip (though this source is not public).

2.4.2 Methodology

I used an iterative and incremental development methodology. Each part of the pipeline described in Section 2.3.3 was implemented in order as required for the end runtime implementation to be meaningfully tested. Each stage would be revised and iterated upon once a future stage revealed a shortcoming in the previous stages.

3 Implementation

The project forms a pipeline from training data acquisition in Maya through to a playable run time (Section 2.3.3). This chapter will describe each stage of the pipeline chronologically in both their placement in that process and when they were implemented during development.

3.1 Environment Setup

3.1.1 Library Building

The first requirement for this setup was the ability to read Alembic geometry files using Python. This can be achieved using the Python bindings for the Alembic library [8]. Achieving this setup for use with Python 3.14 on Windows was not straightforward, requiring several dependent libraries to be rebuilt. The build process for Windows had limited documentation or forum discussion. This process was worth learning in order to provide guidance to other developers using future Python versions in a Windows environment — especially where Python versions may determine GPU support for PyTorch.

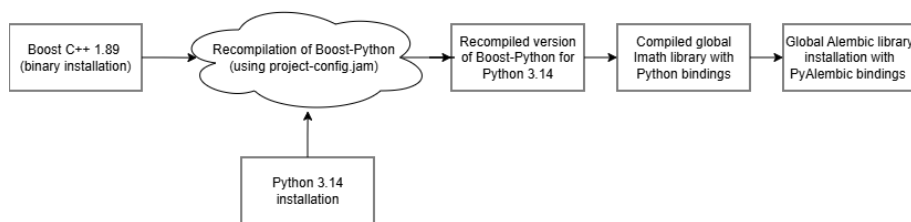


Figure 3.1: Alembic and PyAlembic dependencies.

Alembic depends upon the C++ library Imath which in turn depends upon Boost, a set of general purpose C++ libraries. These dependencies are shown in figure 3.1. To achieve Python 3.14 bindings in Alembic, Boost-Python must be recompiled with a project-config.jam pointing towards the relevant Python installation. Similar config files are required for the Imath and Alembic builds with their associated Python bindings; both pointing to their dependent libraries and the global Python installation.

Notably, the number of .obj sections the Imath and Alembic builds generate require the MSVC /bigobj flag to be set [16].

3.1.2 Python Virtual Environment

The project itself, along with any additional libraries, is contained within a Python virtual environment. Only the Alembic and Imath installs are globally referenced using .pth files pointing to their globally installed binaries and bindings. The dll directories are added in the first line of Python scripts using them.

Libraries initially installed in the virtual environment included NumPy and SciPy for matrix work, PyTorch for NN training and TensorBoard for loss visualisation.

3.2 Training Data Acquisition

To train the ball-cloth model, I needed to generate many continuous frames of cloth vertex positions and ball positions from a physical simulation. This could be acquired using arbitrarily complex offline methods.

3.2.1 Maya Scene Creation

I learnt how to create scenes for physics simulations using built-in Maya tutorials and YouTube videos [18].

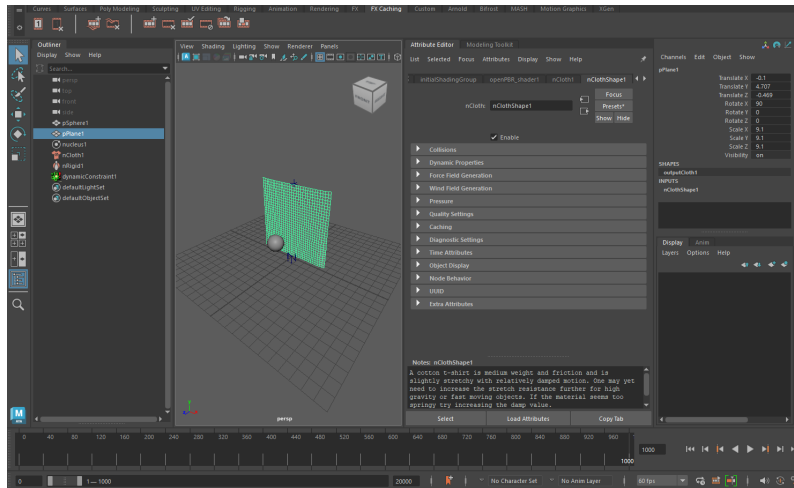


Figure 3.2: Maya cloth and ball scene.

The cloth is a subdivided plane with an attached nCloth instance; the ball is registered as a passive collider. The nCloth can be preset to different materials, I chose to use the t-shirt, in keeping with Ubisoft, and set its thickness appropriately high to limit the chance of self-sticking artefacts. I also set its *Input Mesh Attract* to 0.2 such that the model would return to its stable resting point quicker which aesthetically looked heavier and more substantial. Finally, the cloth was tethered at the top by applying nConstraints to some of its top vertices to hold them in place. The quality of the physics simulation could be tuned in the Maya *Nucleus*, the object responsible for resolving the physical constraints of the scene. This was set to a high

quality of 10 substeps per frame with a max collision iteration of 4. An animation could be created by keyframing different positions of the ball over time. The framerate was set to 60-fps in order to train a model for a standard high framerate game [19].

3.2.2 Geometry Caching in Maya

From an animation of the ball moving around the scene colliding with the cloth, the goal is to export a frame-by-frame cache of the resolved cloth vertex positions and the ball's transform. Geometry caching for Maya physics animations takes place automatically during scene editing around the keyframes animators are working at for temporally local visualisation.

To capture the complete animation, we save an mcx file to disk. This is a Maya-specific geometry cache file type. During cache creation, the animation is played back and the simulation is resolved with the results displayed in the viewport. We can then export the targeted objects included in the simulation as an abc file (Alembic cache) from Maya. This will encapsulate the scene-tree structure and vertex samples through time.

3.2.3 Animation Automation

Maya GUI tasks can be automated using the Maya Embedded Language (MEL) and Python scripting.

I opted to create and export hundreds of smaller animations (each 2 minutes 47 seconds, 10,000 frames) generated from the same key-framing algorithm into a single folder, referred to as a *bank*. This approach crucially allowed me to append to a bank at different points in time and mitigate the effects of animation artefacts; where the cloth would occasionally clip and stick to itself, permanently making the rest of the animation unusable. With 9 hours of animation taking roughly 27 hours to simulate and export, an early artefact could potentially invalidate hours of animations if these were rendered as a single continuous motion. From inspecting loss curves, problematic caches could be identified and inspected rather than having to watch a rendered replay of the whole bank.

A single-file-per-frame approach could be used to circumvent live memory mcx cache-size restrictions (with one large Alembic file being exported and used in training). However, given the shorter animations, I opted for a single file per-cache approach for both mcx and abc files. This naturally provided a clear separation between animations which would later be important for training (Section 3.3.4).

Banks were automatically named and dated to keep track of training sets. This scheme created separate structures for mcx and abc files as shown in Figure 3.3. The generation process included a start-offset for the naming convention for use in appending to existing banks.

I used two methods for animation keyframing; completely randomised, as described in the Ubisoft paper, and a constrained approach. Both execute as shown in Figure 3.4.

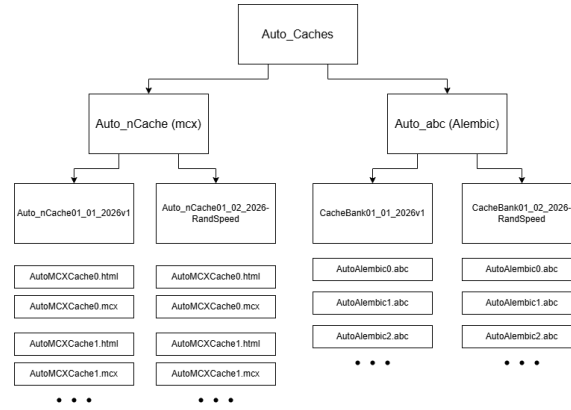


Figure 3.3: Exported cache file structure.

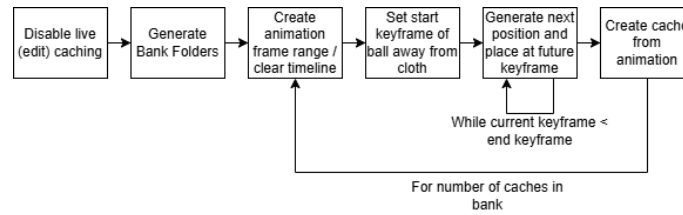


Figure 3.4: The Python/MEL script for procedurally generating animations and exporting geometry caches.

Fully Randomised

```
while CurrentFrame < EndFrame:
    x, y, z = random.uniform(-8, 8), YTranslate, random.uniform(-5, 5)
    CurrentFrame += random.uniform(0, MaxInterval)
    SetSpherePosition(x, y, z, CurrentFrame)
```

Listing 3.1: Fully randomised animation pseudo code.

This method aims to create as many unique interactions as possible [12] by placing random positions at semi-random times as shown in Listing 3.1. This creates a more generalised model. Although, as described in the introduction, this NN approach is intended to benefit constrained gameplay scenarios most, e.g. when the dimensionality is reduced. Player speed, for example, is a DOF that is constrained in many games.

Constrained Speed

```
while CurrentFrame < EndFrame:
    x, y, z = random.uniform(-8, 8), YTranslate, random.uniform(-5, 5)
    Offset = PrevPosition - [x, y, z]
    SquareDist = Dot(Offset, Offset)
    Distance = sqrt(SquareDist)
    MotionSpeed = Speed + random.uniform(-SpeedRange, SpeedRange)
    Frames = (Distance / MotionSpeed) * 60
    CurrentFrame += Frames
```

Listing 3.2: Constrained speed animation pseudo code.

In constrained circumstances, representing this in the training data could create a more accurate model [14]. Each random position generated has a corresponding

random speed of approach which is selected from a range. The number of frames in the future for which a position must be keyframed in order for this speed to be achieved is then calculated using a basic speed-distance calculation as seen in Listing 3.2.

Random Stops

```
test = random.uniform(-10, 240)
if test < 0: # One in 25 chance of staying still for some time
    CurrentFrame += random.uniform(20, 80)
    SetSpherePosition(CurrentX, CurrentY, CurrentZ, CurrentFrame)
```

Listing 3.3: Random stopping pseudo code.

Both methods also use a random stopping mechanism to generate scenarios where the cloth rests on the stationary object or naturally returns to its start state. At each keyframe, there is a $\frac{1}{25}$ chance of a random stop: the current position being keyframed again (Listing 3.3).

3.3 Model Training

In this section, I detail the implementation of the training algorithm outlined in Section 2.1.

3.3.1 Python Project Setup

The training algorithm is contained within *MultiAlembicFileTraining.py*. Other scripts then interface with this API to perform training using specific banks and hyperparameters (e.g. *FullCoveragePerEpochTraining.py*). Trained models were saved within the Python project hierarchy.

3.3.2 Single Cache and Cache Bank Loading

The original SNP training algorithm implied a singular large continuous geometry cache was loaded and frames could be arbitrarily selected [12]. As detailed in Section 3.2.3, the training data was a collection of individual animations. This was considered throughout the training algorithm.

For per-cache (Section 3.3.6) training, global tensors within *MultiAlembicFileTraining.py* held currently loaded and transformed cloth-vertex and ball-position frames. The highest level training function (*CreateModelFromBanks*) would manage the loading of transformed geometry cache data into these; all other functions by convention return NumPy arrays.

Single Cache Loading

As detailed in Section 2.2.4, reading vertex samples from an Alembic file requires interpreting objects within the scene hierarchy as *schemas*. The scene-hierarchy must be traversed from the top-level archive object; this requires knowledge of the exported scene from Maya or using a DAG traversal like depth-first search to output

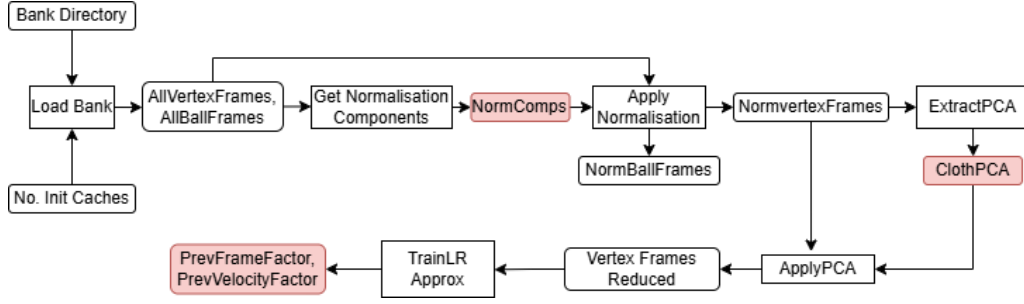


Figure 3.5: Initial model training with saved elements in red.

the scene details. The cloth is read as a *PolyMesh* from which vertex samples can be read using its time-sampling component. Using this, all indexed-samples for the cloth and ball objects can be accessed.

We use information of the first sample to determine the size of the cloth’s frame. In combination with the number of samples stored in the object, we fill an appropriately sized matrix which stores the cloth’s flattened vertex positions per frame (Listing A.1).

Flattened matrices of x, y ball positions (filled by reading the ball object as an *Xform* sampled transform) and vertices are returned.

Multi-Cache Bank Loading

Similarly, we can load multiple caches from a bank for initial training by continuously reading into an extended matrix. This mechanism is used for retrieving data for the initial model creation (e.g. performing PCA). This representative subsample of caches from a bank for initial training is the *initial bank*.

3.3.3 PCA Transformation

We first extract the mean and standard deviation frames for both the cloth and ball from the initial bank, and use these to centre and normalise its values. This ensures unfair priority is not given to components that better describe numerically large components for the cloth (which will undergo PCA).

Due to the unit-length of the cloth, the vertices were centred but did not have to be normalised before the PCA transform in order to yield strong simulation results (Section 4.6). This may not be the case with physically larger objects. SciKit’s PCA method calculates the mean and centres the data as part of its own fitting process - this is the mean that was used.

We extract a specified number of principle components from the transformed initial bank using *SciKit Learn*’s PCA full-solver class which uses the LAPACK method[30]. Finally, we can use the instance of the PCA class we created for extraction to create matrices of fully transformed frames. The size of basis, *NumClothComponents*, is a major determinant in the quality and complexity of the output model. I used $basis \in \{32, 48, 64, 128, 256, 512\}$.

3.3.4 Least-Squares Approximation

The first part of the model is to predict the next transformed-frame based on the approximated inertial movement vectors α and β defined in Section 2.1.3. To calculate these vectors, we must be careful at animation boundaries within the transformed initial bank. While this is currently stored as a continuous matrix for the PCA transform, it is not the case that the first frame of an animation is based on the final two frames of the preceding animation in the bank. We therefore slice the transformed *initial bank* whose animations $a, b, c, \dots$ are currently loaded in a reduced basis of m components as the continuous frames

$$\begin{bmatrix} a_{0,0} & a_{1,0} & \dots & a_{m,0} \\ a_{0,1} & a_{1,1} & \dots & a_{m,1} \\ \vdots & \vdots & & \vdots \\ a_{0,t} & a_{1,t} & \dots & a_{m,t} \\ b_{0,0} & b_{1,0} & \dots & b_{m,0} \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

where $a_{i,t}$ is the i th component of the frame t of the reduced basis animation a . These are sliced to form the following matrices per component

$$\begin{bmatrix} a_{m,1} & a_{m,1} - a_{m,0} \\ \vdots & \vdots \\ a_{m,t-1} & a_{m,t-1} - a_{m,t-1-2} \\ b_{m,1} & b_{m,1} - b_{m,0} \\ \vdots & \vdots \end{bmatrix} \begin{bmatrix} \alpha_m \\ \beta_m \end{bmatrix} = \begin{bmatrix} a_{m,2} \\ \vdots \\ a_{m,t} \\ b_{m,2} \\ \vdots \end{bmatrix}$$

For each component, we concatenate the complete blocks of linear equations generated from each cache starting from the third frame in each. We then run the least-squares calculation per component, generating *PrevFrameFactor* and *PrevVelocityFactor* as α and β respectively.

3.3.5 Full Initial Model Creation

The initial model creation puts together all so-far defined functions shown in Figure 3.5. The bank directory used for initial training should be the same as that used for subsequent network training and the number of caches used in this section should be a representative subsample of all caches. I found 30% would suffice, though the more used, the more accurate to the true principle components of the whole bank and the less error in reconstruction. RAM size was the limiting factor during this process. Alternatively, *IncrementalPCA* could be used for more caches, though this is ultimately also an approximation.

The elements highlighted in red in Figure 3.5 are crucial model components. These will be used throughout training to transform each newly loaded animation into normalised and PCA-reduced frames. These are used at run time to transform out of the subspace: deforming the mesh based on the current PCA coordinates.

3.3.6 Training Data Coverage Techniques

As noted in Section 2.1.3, the corrective NN that is additively applied to the least-squares approximation is trained over windows of the training data. The SNP algorithm described by Ubisoft suggests these are drawn from a continuous large geometry cache [12], however, as motivated in Section 3.2.3 there does not exist one singular continuous animation but instead many smaller ones. Furthermore, as noted in Section 3.3.5 only a subset of all frames can be loaded at any given point.

Mini-Batching

Windows are themselves processed in mini-batches, where the loss over a set of windows is summed and averaged. This is then fed back into the network; this disincentivises over correction for a particular movement. For example, there will be many windows where the ball is moving but not interacting with the cloth and the cloth remains static. Ubisoft suggest using “overlapping windows” in these batches; this technique is implemented in both proposed coverage techniques using window offsets (based on epoch in per-cache methods, Figure 3.10).

Window Extraction from Caches

I developed and trialled two methods for data coverage. First, I used a *per-cache* loading and transformation policy, whereby a full cache would be loaded, transformed and used for training before moving onto the next cache. This itself could be used in a *full-linear* coverage of an entire bank or in a repeated “local-epoch” method where the cache would be used multiple times once loaded with different window offsets. I then took another approach: *total-random*, to take advantage of the efficient loading scheme of Alembic by maintaining an open directory to all caches. In this scheme, windows were enumerated as (CacheID, StartFrameIndex); a list of all of these were generated (with all possible offsets) which could be fetched and transformed individually, allowing for truly random batch selection from the entire training set (this bank-covering window list was covered each epoch in the total-random method).

Both methods share code for data transformation, initial approximation model creation and training over a set of windows.

Window Shifting

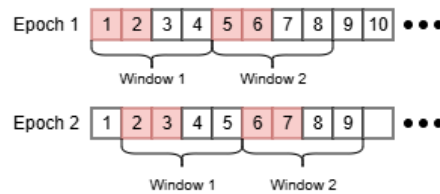


Figure 3.6: Window offset visualisation.

In the *per-cache* method, the windows were shifted after each epoch to create a new training scenario, as determined by the first two frames. The offsets were used to create new possible batches to draw from in the *local-epoch* technique.

Training vs Testing Windows

In the per-cache approach, to avoid excessive stalls from loading specific unseen-caches, a dedicated portion of each would be used as unseen validation data. Every few caches, the currently loaded cache would have its unseen data portion evaluated with the cumulative result logged at the end of the epoch.

In the *total-random* approach, a dedicated section of the whole bank was used for unseen validation as this was no slower than the window loading used for training. This would be evaluated against each epoch.

3.3.7 Total-Random Method

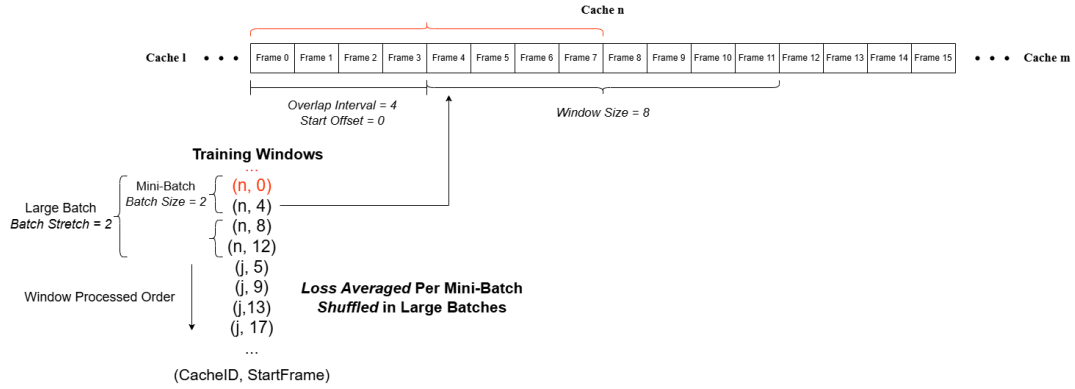


Figure 3.7: Window process order in total-random.

This method draws large overlapping batches of windows at random from the entire training set. To achieve this, all Alembic caches within a given bank are opened and stored in a directory. Each cache is decomposed into its own set of windows with a specified overlap interval. This is an array of indices where each window starts. These are then concatenated and shuffled in batch-sized chunks called “large batches”. This allows for training to take place over n continuous mini-batches from the animation before moving to another large batch from the entire bank as shown in Figure 3.7 with the previously processed window indicated in red. This produces one large list of pairs of window start-indices with a CacheID that align with the batch consumption used in training shown in Listing 3.4. This list is extended with all unique “start offsets”.

This array is sliced into a training and testing set before being shuffled. The training set is processed in batches, feeding each batch’s average loss-per-window into the network.

A cumulative loss over n -batches is also stored and logged for visualisation. This method provides stochasticity in the random selection of large batches from the entire training set with a hypothesised longer training time than the per-cache method due to the per-window extraction and transformation from multiple Alembic files. This was indeed a longer training time; look-ahead loading and transformation however could mitigate this problem.

This approach originally drew windows entirely at random, however this was de-

veloped into the batch-level technique shown in Listing A.2 to ensure all windows within a batch originated from the same animation. This was motivated by the observation that batches composed of windows from multiple caches failed to leave the local minimum of the cloth remaining in its mean position. This was likely as the NN was initially processing too many disparate values.

```

for Epoch in (0, Num_Epochs):
    Training_Windows, Testing_Windows =
        GenerateWindows(BankPath, WindowSize, Offset = Epoch, Ratio= Valid_R)

    # Move through this list of windows in batches
    Num_Batches = len(Training_Windows_Start_Indices/Batch_Size)
    for Batch in (0, Num_Batches)
        Total_Loss = 0
        for Window in (0, Batch_Size):
            Window = Training_Windows[((Batch*Batchsize)+Window)]
            Pos_Loss, Vel_Loss += TrainNetwork(Window_Start, Vertex_Frames)
            Total_loss += Pos_Loss, Vel_Loss
        Total_Loss /= Batch_Size
        Total_Loss.backward() # And set back to 0 etc
        Model_Optimizer.step()
    TestAndLogUnseen(Testing_Windows)

```

Listing 3.4: Pseudo code for the total-random training processs.

3.3.8 Per-Cache Methods

The per-cache method was developed to find a quicker training-time algorithm by loading individual caches completely and transforming them together to amortise the cost. These would then be split into windows and used once in the full-linear approach or multiple times in the *local-epoch* approach, with each local-epoch using a different window offset to create new training scenarios.

Windows in the per-cache methods are simply start-indices referring to frames in the currently loaded cache; created using the returned frames from `LoadBank` (Section 3.3.2).

Local-epoch

Each *GlobalEpoch*, all caches are loaded once in turn for training. When loaded, their training portion (Section 3.3.6) is split into windows, shuffled and then processed in batches like in total-random. This process is then repeated on the same currently loaded cache for a number of local-epochs with an offset determined by both the global and local epoch (Listing 3.5). An unseen-test is then run at intervals of caches using that cache’s currently loaded unseen portion. Training and validation windows remain separated per-cache even if internally shuffled. This is also cumulatively logged after a global epoch for visualisation.

Full-linear

This is analogous to the data-loading mechanism of local-epoch but only ever using exactly one local epoch. Caches are covered in-order with their windows shuffled and offset by the global-epoch.

```

for Global_Epoch in (0, Num_Global_Epochs):
    for Cache_Num in (0, Bank_Size):
        Vertex_Frames = LoadCache(BankPath, Cache_Num)
        CreateTrainingTensors()
        for Epoch in (0, Num_Local_Epochs):
            Training_Windows_Start_Indices, Testing_Windows_Start_Indices =
                GenerateWindows(len(Vertex_Frames), WindowSize, Offset = Global_Epoch *
                               Num_Offsets + CurrentEpoch, Ratio= Valid_R)

            Num_Batches = len(Training_Windows_Start_Indices/Batch_Size)
            for Batch in (0, Num_Batches):
                Total_Loss = 0
                for Window in (0, Batch_Size):
                    Window_Start = Training_Windows_Start_Indices[((Batch*Batchsize)+
                                                                    Window)]
                    Pos_Loss, Vel_Loss += TrainNetwork(Window_Start, Vertex_Frames)
                    Total_loss += Pos_Loss, Vel_Loss
                Total_Loss /= Batch_Size
                TotalLoss.backward()
                Model_Optimizer.step()

            if Batches_Completed % Unseen_Test_Interval == 0:
                LogUnseenLost(Model, Testing_Window_Start_Indices)

```

Listing 3.5: Pseudo code for the local-epoch training process.

3.3.9 Loss Calculation

During training, the first two frames of a window are perturbed by noise (drawn from a normal distribution) and the remaining frames are predicted using these initial two frames and the current model. These predictions are used as the basis for making subsequent predictions until the rest of the window has been predicted as shown in Listing 3.6. The loss is calculated as in Section 2.1.3. The added noise trains the network to return to a seen behaviour throughout the window.

```

WindowPredictions = []
WindowPredictions.append(VertexFrames[0] + FirstFrameNoise)
WindowPredictions.append(VertexFrames[1] + SecondFrameNoise)
for i in range(2, WindowSize):
    ApproximatePrediction = (t_PrevFrameFactor * WindowPredictions[i-1]) + (
        t_PrevVelocityFactor * (WindowPredictions[i-1] - WindowPredictions[i-2]))
    NetworkInput = torch.cat([ApproximatePrediction, WindowPredictions[i-1],
                              ExternalFrames[i]])
    CurrentCorrection = Model(NetworkInput)
    FullPrediction = ApproximatePrediction + CurrentCorrection
    WindowPredictions.append(FullPrediction)
t_WindowPredictions = torch.stack(WindowPredictions)
PosLoss = loss_fn(t_WindowPredictions[2:], VertexFrames[2:])
VelLoss = loss_fn(t_WindowPredictions[2:] - t_WindowPredictions[1:-1],
                  VertexFrames[2:] - VertexFrames[1:-1])

```

Listing 3.6: Window loss calculations from MultiAlembicFileTraining.py.

The loss is calculated as the Mean Absolute Error (PyTorch L1 loss) between the predicted frames and their true transformed representations, and their error in per-frame velocity (motivated in Section 2.1.3).

3.3.10 Hyperparameter Discovery

I trialled different hyperparameters by initially finding a set that produced reasonable results; those from the original SNP paper [12] were able to achieve this. The hyperparameter discovery process was time-limited given the length of the dissertation and training times (one to two days), as well compute needs in other aspects of the Tripos.

Windows I used a window size of 32 and a mini-batch of 16 windows. The noise vector that is added to the first two frames of any window was drawn from a normal distribution with variance 0.0001 - having trialled values in the range 0.1 to 0.00001. **Initialisation** I trialled zeros, Xavier [3] and Kaiming-Uniform [6] initialisation methods. I found Kaiming achieved the best results in per-cache training (Section 3.3.8). **Learning-rate decay** I used an exponential learning rate decay with a gamma of 0.999. This was applied every 1000 mini-batches in both methods. **Initialization Caches** The number of caches in the initial bank had an impact on model performance (noted in Section 3.3.5). **Local Epochs and Total Random Parameters** I found three local-epochs in the per-cache method worked best having trialled 2 to 8. I also found a batch stretch of 64 for the large batches and an overlap interval of 8 in the total-random method led to a steady decrease in loss. **Number and Size of Hidden Layers** The size of the hidden layers were set to 1.5 times that of the reduced basis and there were 10 hidden layers, in keeping with the original paper.

3.3.11 Model Saving and Unity Exporter

Models were stored internally for offline simulations and in an exported format within the Unity project for the runtime implementation.

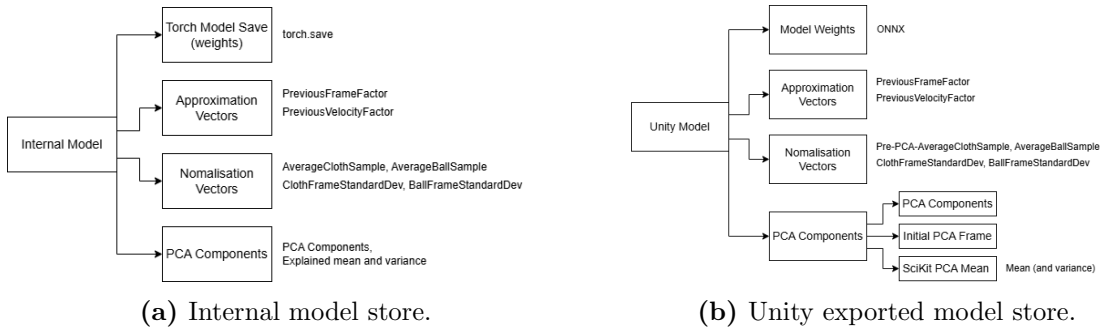


Figure 3.8: Believability study sampler.

The network was saved internally using PyTorch’s saving capability, and vectors using `np.savez`. Related vectors were stored within the same file as a dictionary (Figure 3.8a).

The Unity export script saved the transformed files from the internal save into the target project. The internal PyTorch NN is exported as an ONNX file, which Unity’s *Inference Engine* can use. The library *NumSharp* (used for reading the NumPy vectors in a Unity C# script) could not handle different sized vectors stored within the same dictionary file, nor could it read matrices. Therefore, the PCA file

had to be split into separate mean/variance files and a flattened component file. An initial PCA state for the cloth was also included.

3.3.12 Extension: Offline Simulations and PCA Visualisation

To visualise internal models results quicker, I created an offline abc simulation method allowing for Maya/Blender playback (Listings A.3 A.4). I also created a script for animating each PCA component in turn and exporting as an abc. This was used to determine a sensible level of noise to apply during training for the first two frames [12]. These are shown in *DemoVideo.mp4*.

3.4 Runtime Implementation

I first created a blank project in Unity 6000.2.8f1. I then installed the *Sentis* (*Inference Engine*) 2.4.1 package into the editor and added the NumSharp dlls to the project. After running the Unity export scripts from the Python project, the model files were saved in the Unity project hierarchy (Figure 3.9b).

I created a procedural cloth mesh analogous to the offline simulation method (Section 3.3.12). Triangle face-indices in Unity did not require higher level interpretation as needed in the offline simulator. The mesh is stored in a *MeshFilter* and is displayed by a *MeshRenderer* when attached to the same Unity *GameObject*. I then created a joystick controllable ball.

3.4.1 Model Loading

The model vectors stored are unpacked using the NumSharp C# library and stored in float arrays within the *MLMeshDeformer* class for the CPU side implementation and *MLMeshDeformerControl* for the GPU implementation. This class derives from the Unity *MonoBehaviour*, allowing the mesh-deformer to be attached to an object within the Unity scene.

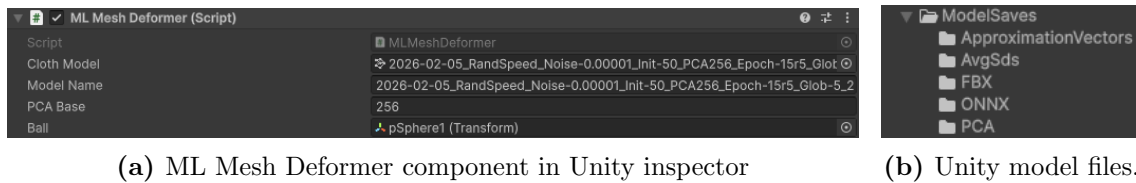


Figure 3.9: Believability study sampler.

The runtime implementation component requires the model name, ONNX file and reference to the in-scene ball object (Figure 3.9a). The NumPy files are opened from the asset files in the editor; for a built game, the models must be stored within the *StreamingAssets* path. The ONNX file is a registered Unity asset so can simply be referenced through a serialisable field in the Unity script. These files are copied directly when the game is built.

3.4.2 Solver Intervals

The runtime implementations are trained to run at 60-fps assuming the models are run once per frame. This is a *solver frequency* of 60, the number of updates performed to a model per second. To ensure this frequency can be set, we run in either `MonoBehaviour's Update` or `FixedUpdate`. The latter (typically used for physics updates) is called with a constant (settable) fixed interval by the engine itself. This allows the model to run at its intended 60-fps while the rest of the game world runs at an uncapped framerate. The former, `Update`, is called once per frame with frames being generated as quickly as possible. We can therefore set the interval for model runs in `Update` in two ways: cap the framerate of the entire game to 60-fps such that `Update` is called 60 times per second, or we run the model in an uncapped framerate but keep track of the time passed since the last model iteration and only run the model if this has been greater than $\frac{1}{60}^{th}$ second (Listing 3.7). This would also allow the rest of the game to run at an uncapped frame rate. The frames on which these calls occur are offset to ensure not every instance runs a model update on the same frame during multi-instance testing.

```
timeBetween = (UnityEngine.Random.Range(0, 100) / 100) * interval;
...
private void Update()
{
    timeBetween += Time.deltaTime;
    if (timeBetween < interval)
    {
        return;
    }
    timeBetween -= interval;
```

Listing 3.7: Update interval model running.

3.4.3 CPU Runtime Solver

The runtime implementation must keep track of the current and previous PCA frame, which is used in the vertex-velocity input to the NN. Both are initially set to the exported first-frame.

Unity's *Inference Engine* is interfaced with by `Workers`. A `Worker` takes a loaded model as an initial input. It can then be given inputs to process and has various synchronous and asynchronous methods for output depending on the backend execution type specified; I use "CPU" (backend multi-threaded) for synchronous implementations and "GPUCompute" for asynchronous implementations where downloads from the GPU would otherwise stall the main thread.

Synchronous

A basic synchronous system is shown in Listing 3.8. The key elements are the initial prediction, normalisation of the ball position, scheduling the worker, applying the NN result correction and updating the state of the stored PCA frames.

The final PCA frame generated is used to deform the mesh using the method shown in Listing A.5, interfacing with Unity's `Mesh` API. For each component of the current PCA frame, we add the total contribution to each vertex by that component.

```

List<float> input = new List<float>();
for (int i = 0; i < PCABase; i++)
{
    float ComponentPred = (PreviousFrameFactor[i] * CurrentPCACoords[i]) + (
        PreviousVelocityFactor[i] * (CurrentPCACoords[i] - PrevPCACoords[i]));
    InitPred[i] = ComponentPred;
    input.Add(ComponentPred);
}
for (int i = 0; i < PCABase; i++)
{
    input.Add(CurrentPCACoords[i]);
}
input.Add((Ball.position.x - AverageBallPosition[0]) * -1 / (BallSD[0]));
input.Add((Ball.position.z - AverageBallPosition[1]) / (BallSD[1]));
// +2 is for ball
using Tensor<float> inputTensor = new Tensor<float>(new TensorShape((2 * PCABase) +
    2), input.ToArray());
ClothWorker.Schedule(inputTensor);
using var outputTensor = ClothWorker.PeekOutput() as Tensor<float>;
for (int i = 0; i < PCABase; i++)
{
    PrevPCACoords[i] = CurrentPCACoords[i];
}
// Get the now ready readback of the correction
float[] InitCorrection = outputTensor.DownloadToArray();
// Apply the correction
for (int i = 0; i < PCABase; i++)
{
    CurrentPCACoords[i] = InitPred[i] + InitCorrection[i];
}
ShapeClothFromPCA(CurrentPCACoords);

```

Listing 3.8: Synchronous runtime model.

Normals are recalculated per frame using Unity’s own implementation. This deformation method runs on the main thread and works with either of the NN inference backends.

Asynchronous

I also created a macro-enabled asynchronous *Inference Engine* querying mechanism in *MLMeshDeformer* to avoid blocking calls when the backend type is *GPUCompute*. The logic for PCA deformation and updating remained entirely the same, though the correction application and PCA deformation existed within a lambda-style callback (Listing A.6).

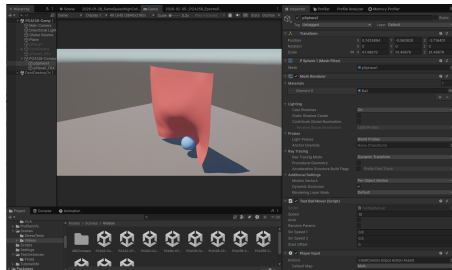


Figure 3.10: Ball controlled via joystick with PCA-128 CPU solver in editor display port.

3.5 Extensions

I implemented a number of extensions to the run time to improve workflow and performance.

3.5.1 FBX Pipeline

I first implemented a mesh pipeline to remove the current procedural generation; meaning the model files would fully encapsulate the soft-body scene. The FBX cloth model is exported along the pipeline from Maya. Unity applies mesh optimisations by default where face and vertex indices are reordered to optimise for rendering. This was disabled as vertex order is mapped directly to the PCA basis and model NN inputs/outputs.

3.5.2 GPU Compute Shader Implementation

To maximise runtime performance, I implemented a HLSL compute shader that semantically implements the same model execution as the CPU solver but on the GPU. Architecturally, the aim was to minimise interaction with the CPU per frame.

Preparation

I used the Unity documentation and YouTube videos to learn about compute shaders. I used the *Time Ghost* sample project [32] to learn how to pin Unity Tensor buffers (used by the *Inference Engine*) to the compute-shader and how to write directly into Unity mesh buffers from within a shader.

Algorithm

Post initialisation, the model data passed between the CPU and GPU is limited to the ball position taken from the `GameObject`'s transform. CPU communication is then limited to dispatch calls and buffer setups for data that already resides on the GPU. This includes current and previous cloth PCA frames, networks inputs and outputs and the mesh's vertex buffer used for rendering.

The queue of GPU interactions is built in a `CommandBuffer` and dispatched from the C# script using Unity's graphics API. Further optimisations are discussed in Section 3.5.3.

During initialisation, the PCA basis transformation matrix and frame normalisation vectors for converting to absolute vertex positions post basis-transformation is transferred to the GPU. This is handled in a C# script that initially reads the model files using `NumSharp`.

All vector methods in the CPU-runtime were transposed into vectorised kernels. The vectorised version of the PCA deformation is shown in Listing A.7; the results are stored directly into the GPU vertex buffer of the mesh itself. The accessing of the mesh buffer was analogous to the method used in the *Time Ghost* sample project [32]. Operations that could be completed together were grouped into the

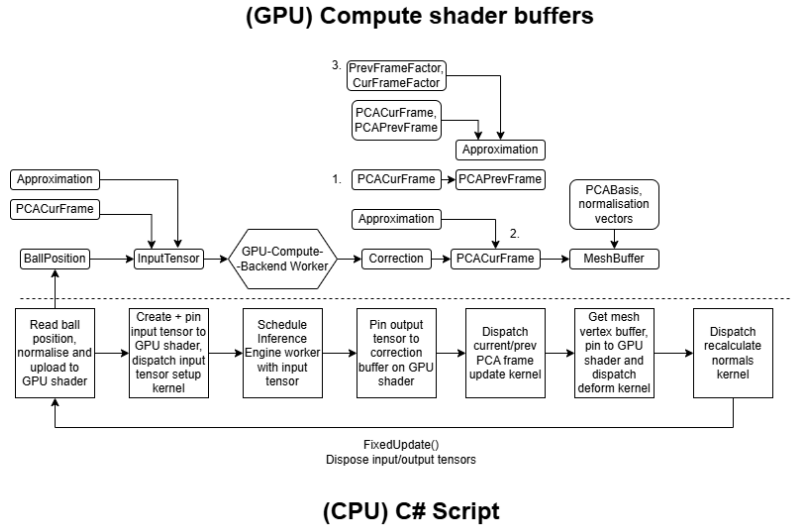


Figure 3.11: Overview of ComputePCADeformer.compute and MLMeshDeformer.cs.

same kernel; this revealed how the next frame’s initial approximation vector could be calculated in the preceding frame alongside the update of the “previous frame” buffer.

All kernel’s are dispatched in groups of 64, except for normal recalculation which is optimised for grid grouping. A single input tensor is maintained throughout the lifespan of the cloth instance, however new tensors are created on each output. These are disposed of at the end of each frame.

Normals and Bounding

After the vertex positions were set, the normals of each would be outdated. These were recalculated using a finite difference method for approximating the partial derivatives at each vertex position on the surface horizontally and vertically with (existing) adjacent vertices. The cross product of these were normalised and written into the vertex buffer at the corresponding offset.

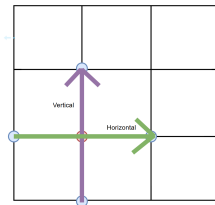


Figure 3.12: Finite difference normals on grid.

The method I used for calculating neighbouring vertices took advantage of the grid layout of the cloth with a known width and using thread grouping. A more generalisable approach for other soft-body meshes would require an adjacency matrix to be built on initialisation. This regular grid layout could be assumed from the Maya export as mesh optimisations had been disabled for PCA alignment noted in Section 3.3.12.

The bounding box for use in frustum culling also needed to accommodate the mesh as sized in the compute shader. These bounds were set as hard-coded maximums.

3.5.3 Aesthetic Dampening

As discussed in Section 3.3.3, the vertex data did not have to be normalised in order to yield strong simulation results. In place of this cut operation in the run time, I multiplied the PCA vertex-velocity by a dampening of 0.975 before being used for the initial prediction. This arose from the observation that even when a target framerate was set in Unity, this was often exceeded by a few frames or would oscillate. I discovered that in order to adjust for this jitter, the perceived movement of the cloth to the NN itself could be dampened. As shown in the user study, this did not significantly decrease the believability of the model compared to the initial Maya simulations (Section 4.6).

Optimisations

```
int vertex_start = (3 * vertex_index);
for (uint i = 0; i < PCABasisSize; i+=1)
{
    float PCACoord = PCACurFrame[i];
    deformed_position.x += PCABasis[(i * AbsoluteVertexFrameSize) +
        vertex_start] * PCACoord;
    deformed_position.y += PCABasis[(i * AbsoluteVertexFrameSize) +
        vertex_start + 1] * PCACoord;
    deformed_position.z += PCABasis[(i * AbsoluteVertexFrameSize) +
        vertex_start + 2] * PCACoord;
}

Vs.

uint VertStart = PCABasisSize * vertex_index;
for (uint i = 0; i < PCABasisSize; i+=1)
{
    float PCACoord = PCACurFrame[i];
    deformed_position += PCABasis[VertStart + i] * PCACoord;
}
```

Listing 3.9: Using non-transposed vs. transposed PCA matrix in shader.

To optimise per-thread performance during the GPU implementation's PCA deformation, I transposed the PCA transformation matrix such that each thread accessed a continuous section of memory. This operation is shown in Listing A.8. This grouped each per-vertex contribution from each PCA base into a single row of the matrix; as opposed to each row being a whole PCA base. This would allow for memory locality to be taken advantage of when threads fetch the matrix during the transformation for-loop due to a much reduced stride length.

For PCA components $l, m, \dots$ align their contributions to each vertex per row into a `float3` (or in Unity terms, a `Vector3`) into the matrix given as

$$\begin{bmatrix} (l_{v1,x}, l_{v1,y}, l_{v1,z}), (m_{v1,x}, m_{v1,y}, m_{v1,z}) \dots \\ (l_{v2,x}, l_{v2,y}, l_{v2,z}), (m_{v2,x}, m_{v2,y}, m_{v2,z}) \dots \\ \vdots \end{bmatrix}$$

Each vertex contribution is then multiplied by the respective current PCA coordinate of the cloth. This alignment gives way to the use of `float3` operations rather than three separate operations per-vertex within the inner loop of the deformation kernel (comparison in Listing 3.9).

I also moved the normalisation of the ball position into its own kernel, ensuring normalisation was applied by multiplying by a constant precomputed $\frac{1}{\sigma}$ (as opposed to using expensive per-thread division).

3.5.4 Multithreading with the Unity Job System

After initial profiling, I discovered the CPU implementation spent over half its time in the main player-loop `Update` call deforming the mesh from the newly calculated PCA coordinates. To improve the performance of this operation on the CPU I multi-threaded the code using Unity’s *Job System* (a feature I had never used before). “Jobs” are written from a per-vertex perspective as structs which inherit from `IJobFor`. Jobs are then distributed across worker threads, taking advantage of a work-stealing system to spread the workload evenly. I directly updated the mesh’s vertex buffer using a `NativeArray` as the target buffer to avoid using copy operations where possible. This required `Unsafe Code` to be enabled in the Unity preferences.

```
NativeSlice<byte> VertSlice = VertexBuffer.Slice(i * VertexBufferStride, 12); //
    floats are 4 bytes, so float 3 is 3*4
void* VertSlicePtr = VertSlice.GetUnsafePtr();
UnsafeUtility.CopyStructureToPtr(ref vert, VertSlicePtr);
```

Listing 3.10: Native buffer writing from thread Job.

In order to write the `float3` vertex into the buffer, a sliced view of the `NativeArray` was created starting at the location of the vertex with a stride length equal to the size of the `float3`. There is no offset as the position is the first element of each vertex in the buffer. These jobs were issued in batches of 128 as shown in Listing A.10. From the documentation, the *Inference Engine* itself schedules successive NN executions with job dependencies in the backend but these are not exposed. As such, the job runs on the “default” dependency. The mesh is flagged for frequent updates to the buffer.

3.6 Evaluation Tooling

To avoid conflicting file reads in multi-instance testing, I created static fields in each of the CPU and GPU `MonoBehaviour` scripts to store the data read out of the model files.

MLMeshDeformer and *MLMeshDeformerControl* (and its issued command buffer) were instrumented with profiling calls. I implemented a framerate calculator with scene specification settings (e.g. capping global framerate) and a ball movement script to generate consistent playbacks for the evaluation. This oscillated the ball on both the x and z components.

3.7 Repository Overview

All training environment scripts are found within *PythonSource*. The main training file is *MultiAlembicFileTraining.py*. This is used by *FullCoveragePerEpochTraining.py* demonstrating the use of the per-cache training mechanism and saving the resulting model internally. *PCATestAnim.py* is used for exporting Alembic files illustrating the modes of deformation and *LoadAndSimulateModel.py* is used for creating offline Alembic animations using internal models. *UnityExportModel.py* exports an internal model, which were saved in *ModelSaves*, to the Unity project. Other files show single-file simplified components of *MultiAlembicFileTraining.py*. *AutoSphere-Animate* is the script ran inside Maya for bank generation (Section 3.2.3).

The Unity scripts are found under *UnitySource*. Key scripts here are *MLMeshDeformer.cs* for the CPU bound implementation and *MLMeshDeformerControl.cs* and *ComputePCADeformer.compute* for the GPU implementation.

DemoVideo.mp4 shows the PCA visualisior, offline renderer and Unity run time videos included (unlabelled) in the survey of believability (Section 4.6).

4 Evaluation

All tests were performed on a gaming PC with an AMD Radeon RX 9070 XT (16 GB) graphics card, AMD Ryzen 7 9800X3D 8-Core Processor (4.70 GHz) processor and 64 GB RAM.

4.1 Models and Runtime Implementations

The models evaluated were trained with reduced basis PCA-32, PCA-64, PCA-128, PCA-256 and PCA-512. All models were trained on the same 200 caches using the per-cache method. All but PCA-256 and PCA-512 were trained using the same number of local and global-epochs using the per-cache methods as these would have taken an unachievable amount of time to train with the same coverage as outlined in Section 4.7. PCA-512 was only used for performance evaluation which did not require a visually believable model.

There are two categories of runtime implementation: CPU bound and compute shader. The shader can only use the *Inference Engine*'s GPU backend execution and only has one method of deformation (the process of taking a PCA frame, converting this into a set of absolute vertex positions and applying this to the mesh), that which also occurs on the GPU.

The CPU bound implementations are tested against CPU and GPU-Compute *Inference Engine* (Sentis) backends. There are also two methods of deformation for the CPU-bound implementation, the single threaded method which uses Unity's mesh vertex interface and the multi-threaded method using Unity's Job System.

Comparators

The Unity runtime implementation (*Cloth component*) uses a different algorithm to that in the Maya offline simulations, giving it a distinct appearance. It also runs at a settable *solver frequency*. It is a useful comparison point when running at the same frequency as the SNP implementations as it is the only in-built Unity-maintained runtime implementations developers can use.

The Ubisoft per-frame breakdown results of their SNP implementation of the ball-cloth model are also compared against in Section 4.4.1.

4.2 Uncapped Model Performance

This section analyses the theoretic maximum solver frequency a model could be trained to target and be used within a Unity scene. These tests were conducted in the editor using tools from Section 3.6 with a maximised display resolution to model built-game behaviour more accurately. Models were called from `Update` with no set interval, meaning they are ran as frequently as possible. Framerate in this test is therefore equal to the solver frequency.

Tests using uncapped framerates make the cloth move rapidly. However, the information provided by these tests paint a picture of the overall performance of the given implementation. A model could then be trained to target these maximum framerates and these would visually move at the correct speed.

4.2.1 Single Instance

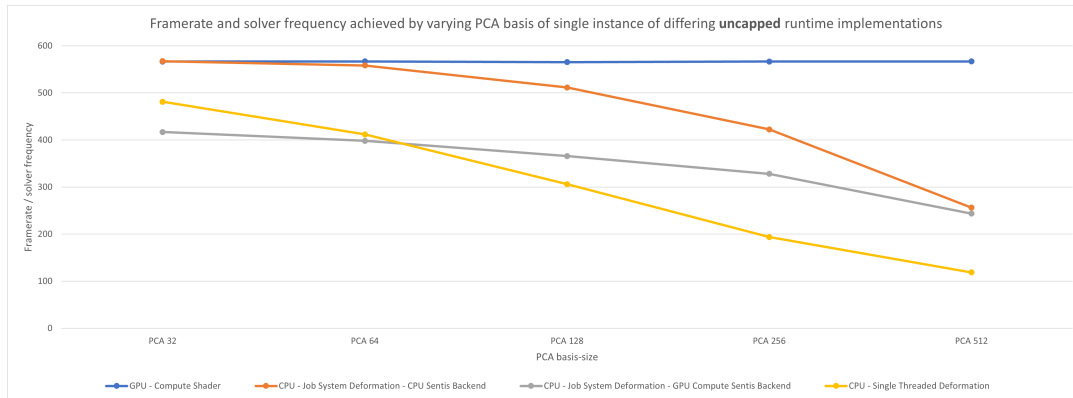


Figure 4.1: Framerate of single instance achieved by each implementation.

Figure 4.1 shows per-implementation performance as PCA-basis varies, demonstrating the average framerate and solver frequency over 90 seconds commencing from the moment the scene enters “Play Mode” in editor.

The GPU compute shader is the most performant across all PCA basis sizes and scales constantly up to PCA-512, running at 695.4 across all PCA basis sizes. Both multi-threaded deformation implementations are more performant than their single threaded counterpart for a basis sizes greater than 32.

For the multi-threaded CPU deformers, executing the inference natively on the CPU is much more performant for lower PCA basis sizes. However, for PCA-512, the cost of offloading the inference to the GPU and asynchronously downloading back becomes worth it compared to executing such a large inference on the CPU.

All methods perform better than the intensive Maya implementation which is limited at 39.7 fps. This means the uncapped compute shader implementation shows a $17.5\times$ improvement in performance over the algorithm used to generate its training animations.

4.2.2 Multi Instance

All cloth instances were rendered in the same location with the camera stationary to account for Unity’s Level of Detail optimisations.

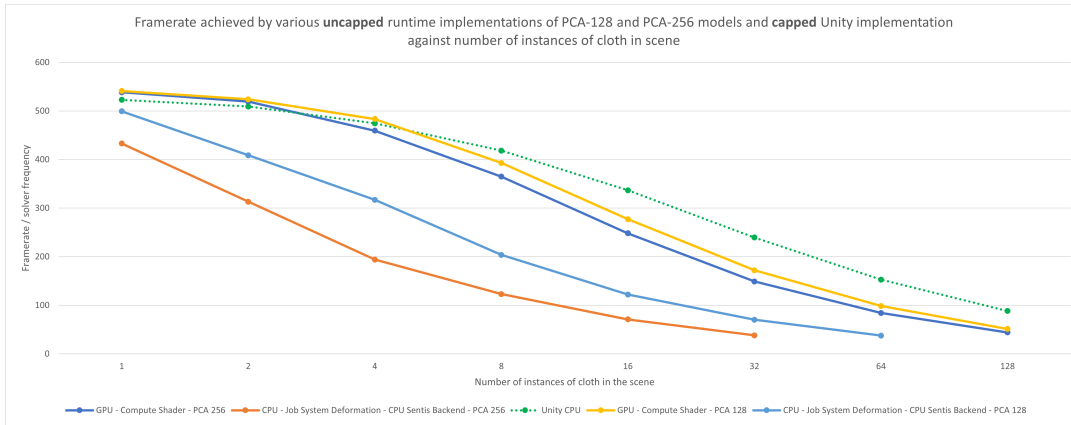


Figure 4.2: Framerate achieved by implementations of SNP PCA-128 and PCA-256 as number of instances of cloth increased against Unity CPU implementation.

The two compute shader performances scale equivalently with the number of instances as shown in Figure 4.2. These uncapped models following a similar scaling trend to the the fixed interval Unity solver as a guide.

The performance loss from using a higher PCA basis is more prominent when the deformation takes place on the CPU, as shown by the increased gap between the PCA-128 and PCA-256 job system instances compared to the two compute shader implementations.

The maximum framerate achieved for any singular instance model here in the multi-instance scene is lower because of spawner management.

4.3 Capped Model Performance

This section looks at the capped solver-frequency runtime implementations that align with the execution pattern of Unity’s own cloth component. This provides a comparison of performance with implementations running at the same frequency. The models here are ran at a solver-frequency of 60 updates per second — the

frequency the SNP models have been specifically trained for. The Unity implementation’s solver frequency is also set to this.

The performance of fixed interval models is measured by global framerate achieved as an abstraction over many composite elements that the Unity cloth component relies upon. Analysing the performance via profiler frame analysis for comparison to the Unity implementation would have to consider many more components of the engine such as collider checks, physics casts etc... which would require an in-depth knowledge of the Unity engine backend. Unity is close sourced; we therefore use global framerate to abstract over this.

4.3.1 Single Instance

Tests were performed over 90 seconds of scene “Play Mode”.

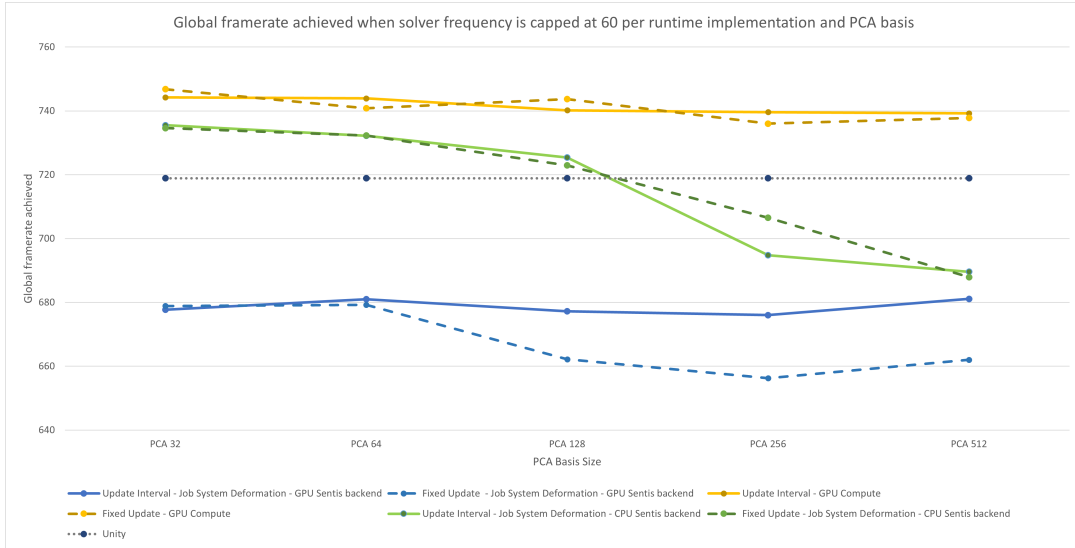


Figure 4.3: Global framerate achieved for SNP models and implementations per PCA basis against Unity implementation, both at solver frequency 60 updates per second.

As shown in Figure 4.3, the global framerate achieved by the SNP compute model on all PCA basis size surpasses that achieved by the Unity implementation. This is also true for the multi-threaded CPU bound implementation up to PCA-128.

The method of fixed interval syncing, **FixedUpdate** or **Update** with basic time checking, generally follow the same performance trends and values as shown in Figure 4.3. This is however not the case for the NN GPU backend where the spaced **Update** performs roughly 20-fps faster. The Unity benchmark shown does of course not use the concept of PCA, but it does target the exact same mesh (with the same number of vertices).

4.3.2 Multi Instance

As shown in Figure 4.4, the compute shader implementation is the most performant for multi-instance as well as single-instance use. This implementation surpasses the

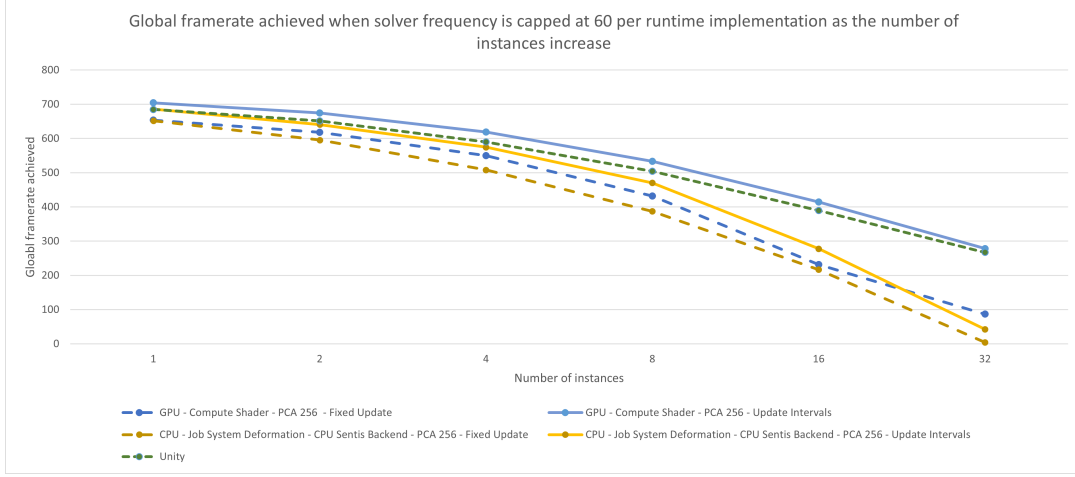


Figure 4.4: Global framerate achieved for SNP implementation against Unity implementation, both at solver frequency 60 updates per second as number of instances increase.

Table 4.1: Time in *ms* and percentage of total processing time per-frame for the CPU-backend/multi-threaded-deformation runtime per model.

PCA	Total	Inference	Approx/Input	Update PCA	Normals	Job
32	0.16	0.13, 81.25%	0.01, 6.25%	0.01, 6.25%	0.01, 6.25%	0.186
64	0.2	0.15, 75%	0.03, 15%	0.02, 10%	0.01, 5%	0.361
128	0.36	0.26, 72.22%	0.06, 16.67%	0.03, 8.33%	0.01, 2.78%	0.711
256	0.56	0.37, 66.07%	0.12, 21.43%	0.06, 10.71%	0.01, 1.79%	1.41
512	0.91	0.55, 60.44%	0.24, 26.37%	0.12, 13.19%	0.01, 1.10%	2.81

global framerate achieved by the Unity implementation for all trialled number of instances when called from intervalled **Update**. These do however begin to converge at $N=8$. Other implementations show significantly worse performance from this point on.

4.4 Profiler Frame Analysis

This per-frame analysis is conducted on the two most performant runtime implementations identified in 4.2: the compute shader and the CPU-NN-backend/multi-threaded-deformation. The profiling information was accessed using Unity’s *Profiler* and *Profile Analyzer* package to draw statistics from 300 continuously recorded frames of run time. The framerate of these runs was much lower due to the profile overheads.

4.4.1 CPU-backend/multi-threaded-deformation Frame Analysis

Table 4.1 shows the per-frame breakdown of mean work spent on each section of the runtime pipeline in milliseconds. The “Job” column shows the mean time a thread takes to execute its portion of all vertices to deform. This does not use a transposed matrix - although this optimisation could be implemented in the future.

Table 4.2: Time in μs and percentage of total GPU time per-frame for the compute shader runtime per model.

PCA	CPU	GPU	Infer	Nrm-Ball	Inp	Corr	Def	Norms
32	56.86	63.07	41.56	9.55	3.96	3.54	2.67	1.79
64	56.64	62.54	41.01	9.76	3.97	3.46	2.57	1.77
128	57.02	63.91	42.34	9.7	3.93	3.54	2.61	1.79
256	56.27	62.45	41.23	9.7	3.83	3.43	2.54	1.72
512	55.21	62.54	41.33	9.56	3.74	3.53	2.59	1.79
Avg	56.4	62.90	41.49 66%	9.65 15%	3.89 6%	3.50 6%	2.60 4%	1.77 2%
Ub-64	38	42						
Ub-128	121	77						
Ub-256	446	157						

Initially, most of the time spent is NN inference. However, as PCA basis size increases, a greater proportion of the work done is in tensor set-up for inputs and updating current PCA coordinates which scales linearly. This operation could be multi-threaded in future implementations to potentially improve performance.

4.4.2 Compute Shader Frame Analysis

Table 4.2 shows the per-frame breakdown of time in microseconds spent on the CPU issuing the CommandBuffer and the time spent on each kernel in the GPU. The time spent in each section shows constant performance as the PCA basis size increases from 32 to 512. Also shown is the Ubisoft results, crucially on older hardware and in their close-sourced engine. The obvious difference is the lack of constant scaling in their implementation and the large proportionate difference between time on CPU and GPU, this could partially be due to their extra work preventing clipping (Section 4.8).

Time spent on the CPU main thread (issuing the CommandBuffer) is 10% of that of the CPU-backend/multi-threaded-deformation implementation. Normalising the ball position is clearly an area for future optimisation. Most work (66% GPU time) is spent on NN inference.

4.5 Memory

The tests of managed heap memory (Figure 4.5) were snapshots taken 30 seconds into scenes with uncapped SNP solver frequencies and high Unity solver frequencies. After each test, the editor was restarted to mitigate for build up of editor memory allocation between tests.

The memory usage for all models and implementations was roughly the same for a single instance, for the PCA-256 GPU shader up to 4 instances and for both PCA-128 implementations up to 16 instances. Both implementations using the PCA-256 model increased memory usage more rapidly as the number of instances grew compared to lesser models. For each model, the runtime implementation that used

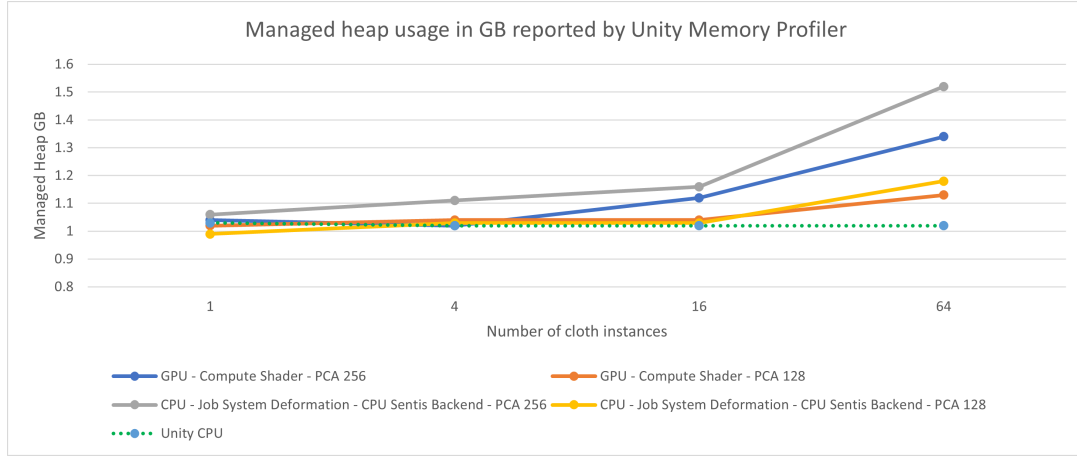


Figure 4.5: Managed heap size for different cloth implementations and models as number of cloth instances increase.

the least memory varied: the PCA-256 model had greater memory efficiency when implemented using the compute shader, whereas this was the other way around for the PCA-128 model.

The Unity implementation had almost constant visible memory usage as the number of instances grew, unlike the SNP implementations.

4.6 User Study

Any performance improvement from the SNP method is only meaningful if no visual believability is lost during the training process. We therefore assess the believability of the runtime SNP method against the visuals of the original simulations it was trained to behave like.

4.6.1 Survey

Participants were told they were going to see videos of implementations for a *heavy drape material* in a game (Appendix A.2). The wording of the material was used to accurately describe the parameters for the soft body that had been set in Maya.

They were shown pairs of unlabelled videos of cloth-ball models in the run time, each 45 seconds. The ball followed the same path in both and the pair was shown in a random order. At each new implementation shown (labelled as letters, e.g. Implementation A), participants were asked to rate it on a scale of one to five from unbelievable to believable. They were then asked to select which of the two was more believable or if they had the same believability. This was supplemented by a text input to explain their reasoning.

Due to university IT Qualtrics issues, I was forced to use Google forms. While this could shuffle the order of videos within pairs, it could not shuffle the orderings between them. I therefore created four forms with orderings taken from a Latin Squares set in order to mitigate ordering effects [10]. A link was then shared between undergraduates which evenly distributed traffic to each. There was a small voucher-

draw incentive.

Models and Videos

Three SNP models were featured in the videos, PCA-64, PCA-128 and PCA-256, as well as a randomly generated Maya training animation. The PCA-256 model appeared twice under different names to ensure internal validity. The scene used for all videos was identical (Figure 4.6a).

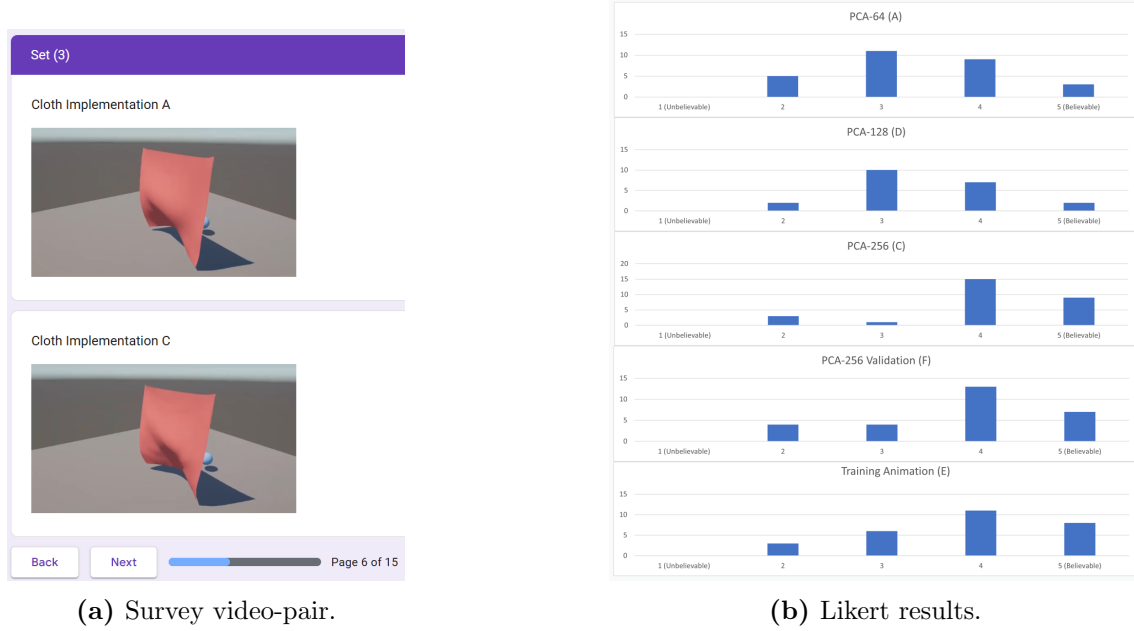


Figure 4.6: Believability study sampler.

The four video pairs were PCA-64 vs PCA-128, PCA-128 vs PCA-256, PCA-64 vs PCA-128 and training vs PCA-256.

4.6.2 Results

28 participants answered. As shown in Figure 4.6b, the PCA-256 implementation was internally valid by comparison of the distributions of (C) and (F) with mean answers of 4.07 and 3.82. These distributions were comparable to the training animation (E) with mean answer 3.86.

The answers received from the direct comparison question per-pair are shown in Table 4.3. N=15 believed the SNP method looked more believable than the training animation itself, with N=10 believing the converse and N=3 believing they were the same.

In a binomial test (ignoring “Both Equal” answers) for whether the PCA-256 implementation is *more* believable than the training animation and where the null hypothesis is that they are both equally believable, the p-value is 0.212. This is not statistically significant. However, it was only desirable that this method results in a material which looks as believable as the training animation which this clearly demonstrates.

Table 4.3: Direct comparisons between implementations of which is more believable.

PCA-256 (F)	Both Equal	Training (E)
15	3	10
PCA-64 (A)	Both Equal	PCA-128 (D)
7	9	12
PCA-128 (D)	Both Equal	PCA-256 (C)
3	1	24
PCA-64 (A)	Both Equal	PCA-256 (C)
5	2	21

There is no statistical significance between the believability of PCA-128 and PCA-64 (p-value 0.180). However, there *is* statistical significance that PCA-256 is more believable than PCA-128 and PCA-64 with p-values of 2.462×10^{-5} and 1.247×10^{-3} respectively.

4.7 Training

**Figure 4.7:** Loss visualisation.

The PCA-32, PCA-64 and PCA-128 were all trained using the same local and global epochs using the per-cache method on the same bank of 200 caches. These each took roughly two days to train, and the training data itself took 27.5 hours to acquire. PCA-256 and PCA-512 would have taken significantly longer for the same coverage, with the PCA-512 model projected to only complete over 8 days. This was partially due to the lack of an NVidia GPU so all training was performed on the CPU. Therefore, the PCA-256 model used was cut short once the loss and unseen-loss curve had mostly flattened.

The training data moved the ball at random speeds between 5 and 19 metres-per-second. The ball in the runtime then moved at a speed between 5 and 7 metres-per-second. This strategy was developed in response to the clipping problem outlined in the following qualitative evaluation.

4.8 Qualitative Comparison

Please refer to `DemoVideo.mp4` for animated examples.

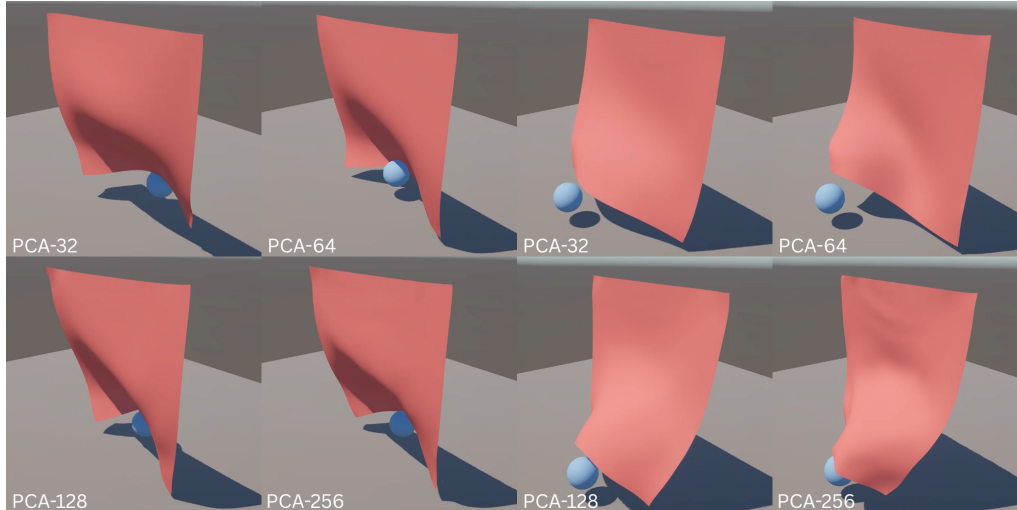


Figure 4.8: Visual comparison of PCA basis.

As PCA basis size increases, the theoretic level of detail captured by the model should increase. The PCA basis is synonymous with the number of DOFs with which the cloth can move. These DOF capture the major modes of deformation, so finite wrinkles in the cloth should only become apparent with sufficiently high PCA basis sizes once the areas of greatest change at the base of the cloth have been captured. This is evident in Figure 4.8 where creases in the top section of the cloth are only visible in PCA-256. For the preceding basis sizes, the cloth appears smooth but better moulds to the position of the ball with increasing basis size. As shown in the video, PCA-32 fails to realistically mould to the ball and significant clipping occurs. PCA-64, while equally smooth, at least reacts to the ball in most contact locations.

The solution used by Ubisoft for clipping was a shader using proxy colliders. However, by targeting training data for use of a larger and faster moving ball, the runtime was unlikely to show these issues for the simple case of ball oscillation.

Detail change was mentioned by four participants in the user study. Comparing the PCA-256 and PCA-64 models, PCA-256 had “more creases”, “ripples”, had “secondary folds”, “folded in more places” and “left imprints of the ball collision”.

5 Conclusion

This project was a success. I successfully engineered an end-to-end implementation of a playable ball-cloth scene using the SNP technique. It meets the success criteria of gathering vertex data, training the required NN and creating a playable run time in Unity. This meets the functional criteria via the qualitative demonstration of the playable scene and meets the non-functional criteria of being more performant (using the GPU compute model up to PCA-512) than Unity’s own cloth component in the single and multi-instance case and being as believable as the Maya simulations.

There are many factors that can be adjusted in the runtime implementations of the models that affect both the performance and believability, including the PCA basis

size of the model itself. It has been demonstrated how the SNP implementations can spread their workload across compute resources; giving developers greater freedom during per-frame budgeting.

Most importantly, the ability for these models to be run exclusively on the GPU is a significant development for soft body simulation in Unity, something not currently possible in either Unity’s own provided cloth component or the popular paid-for asset, *Magica Cloth*. The compute model developed shows comparable performance to Unity’s CPU-based implementation for single and multi-instance use (Section 4.2). It performs $17.5\times$ faster than the cloth algorithm that generated animations upon which the models were trained while retaining the same believability (Section 4.6). Memory efficiency for the runtime implementations is comparable to Unity’s for up to 16 instances (for PCA-128), and up to four instances for the compute shader (for PCA-256). Optimising memory scaling past this point is an area of future development for the SNP runtime implementations.

I have shown how the native CPU runtime implementation can be improved by using the Unity Job System to multi-thread the work of deforming the mesh from PCA coordinates. In addition, I have evidenced how offloading NN inference to the GPU can lead to performance gains for higher PCA basis sizes (Figure 4.1) and how the use of fixed interval `MonoBehavior Update` calls can lead to scene performance improvements for multiple instances (up to 16, Figure 4.4).

There are multiple directions for further development. Instead of training models to target specific framerates, could the framerate itself be included as a model parameter? I developed a pipeline from Maya to Unity whereby new cloth models simply required a few drag-and-drops in Unity before being deployable in-game. This file structure could be developed further such that arbitrary soft-body scenes could be imported into Unity and be playable with the same simple set-up. This would require the inclusion of meta information and a general normal recalculator for the compute model.

If I were to complete this project again, I would implement multiple non-ML implementations to provide a more comprehensive overview of the limitations of current techniques and performances, as well as against *Magica Cloth*. I would also include multiple materials in the evaluation to assess which ones the models can believably mimic better than others and would have performed a thematic analysis on the textual survey answers. I would also use an NVidia GPU given its high performance in ML training.

Overall, this project successfully demonstrates the performance improvements the SNP technique achieves over Unity’s own implementation when deployed in-engine. Unlike typical runtime cloth, including Unity’s, the artistic freedom offered using the SNP method is far superior whereby any material, regardless of computational expense, could be rendered offline in Maya and, with little overhead, brought into Unity with potentially no loss in believability. The runtime implementations provide many options for workload sharing of both NN inference and mesh deformation. Therefore, with the same general execution model, developers can tailor the quality and performance of the SNP soft bodies to any target platform based on its available resources, thus fulfilling this project’s goals.

Bibliography

- [1] Mark Norman and Tim Taylor. “Application of physics engines in virtual worlds”. In: *Proceedings of SPIE - The International Society for Optical Engineering* 4665 (Mar. 2002). DOI: 10.1117/12.458775.
- [2] Edilson de Aguiar et al. “Stable spaces for real-time clothing”. In: *ACM Trans. Graph.* 29.4 (July 2010). ISSN: 0730-0301. DOI: 10.1145/1778765.1778843. URL: <https://doi.org/10.1145/1778765.1778843>.
- [3] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Yee Whye Teh and Mike Titterton. Vol. 9. Proceedings of Machine Learning Research. Chia Laguna Resort, Sardinia, Italy: PMLR, 13–15 May 2010, pp. 249–256. URL: <https://proceedings.mlr.press/v9/glorot10a.html>.
- [4] Benjamin Kenwright, Richard Davison, and Graham Morgan. “Real-Time Deformable Soft-Body Simulation using Distributed Mass-Spring Approximations”. In: *Proceedings of The Third International Conference on Creative Content Technologies (CONTENT)*. 2011. URL: <https://api.semanticscholar.org/CorpusID:9907175>.
- [5] Kathrin M. Gerling et al. “The Effects of Graphical Fidelity on Player Experience”. In: *Proceedings of International Conference on Making Sense of Converging Media*. AcademicMindTrek ’13. Tampere, Finland: Association for Computing Machinery, 2013, pp. 229–236. ISBN: 9781450319928. DOI: 10.1145/2523429.2523473. URL: <https://doi.org/10.1145/2523429.2523473>.
- [6] Kaiming He et al. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *CoRR* abs/1502.01852 (2015). arXiv: 1502.01852. URL: <http://arxiv.org/abs/1502.01852>.
- [7] Ian T. Jolliffe and Jorge Cadima. “Principal component analysis: a review and recent developments”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 374 (2016). URL: <https://api.semanticscholar.org/CorpusID:20101754>.
- [8] Sony Pictures Imageworks Inc. and Industrial Light & Magic a division of Lucasfilm Entertainment Company Ltd. *Introduction Alembic 1.7.0*. <https://docs.alembic.io/python/examples.html>. Accessed: 31-03-2026. 2016.
- [9] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. “On the Convergence of Adam and Beyond”. In: *International Conference on Learning Representations*. 2018. URL: <https://openreview.net/forum?id=ryQu7f-RZ>.
- [10] John T.E. Richardson. “The use of Latin-square designs in educational and psychological research”. In: *Educational Research Review* 24 (2018), pp. 84–97. ISSN: 1747-938X. DOI: <https://doi.org/10.1016/j.edurev.2018.03.003>. URL: <https://www.sciencedirect.com/science/article/pii/S1747938X18301623>.

- [11] Jan van Valburg. “Automated Testing and Profiling for Call of Duty”. Game Developers Conference 2018. 2018. URL: <https://www.youtube.com/watch?v=8d0wzyiikXM>.
- [12] Daniel Holden et al. “Subspace Neural Physics: Fast Data-Driven Interactive Simulation”. In: *SCA '19: Proceedings of the 18th annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Ed. by Stephen N. Spencer. Los Angeles, CA: Association for Computing Machinery, July 2019, pp. 1–12.
- [13] Daniel Holden. “Machine Learning, Physics Simulation, Kolmogorov Complexity, and Squishy Bunnies”. Game Developers Conference 2020. 2020. URL: https://www.youtube.com/watch?v=sUb0W5_waRI.
- [14] Vladimir Vapnik and Rauf Izmailov. “Complete statistical theory of learning: learning using statistical invariants”. In: *Proceedings of the Ninth Symposium on Conformal and Probabilistic Prediction and Applications*. Ed. by Alexander Gammerman et al. Vol. 128. Proceedings of Machine Learning Research. PMLR, Sept. 2020, pp. 4–40. URL: <https://proceedings.mlr.press/v128/vapnik20a.html>.
- [15] Christopher Lewin. “Swish: Neural Network Cloth Simulation on Madden NFL 21”. In: *ACM SIGGRAPH 2021 Talks*. SIGGRAPH '21. Virtual Event, USA: Association for Computing Machinery, 2021. ISBN: 9781450383738. DOI: 10.1145/3450623.3464665. URL: <https://doi.org/10.1145/3450623.3464665>.
- [16] Microsoft Visual Studio 2022 Documentation. */bigobj (Increase Number of Sections in .Obj file)*. <https://learn.microsoft.com/en-us/cpp/build/reference/bigobj-increase-number-of-sections-in-dot-obj-file?view=msvc-170>. Accessed: 24-03-2026. 2021.
- [17] Sean Duffy. *Ultimate guide to profiling Unity games*. Unity Software Inc., 2022. <https://unity.com/resources/ultimate-guide-to-profiling-unity-games>.
- [18] Kurt B. *Maya nCloth: A Comprehensive Introduction*. <https://www.youtube.com/watch?v=EVAmjPdxLaw>. Accessed: 24-03-2026. 2023.
- [19] ASUSTeK Computer Inc. *A Guide to FPS in Gaming - Why Is it Important?* <https://rog.asus.com/in/articles/gaming/what-is-fps-in-gaming/>. Accessed: 15-04-2026. ASUSTeK Computer Inc., Nov. 2025.
- [20] Bryant Francis. *What’s the real reason games are taking longer to make?* <https://www.gamedeveloper.com/production/what-s-the-real-reason-games-are-taking-longer-to-make->. Accessed: 03-04-2026. 2025.
- [21] Giang Tong. “LEGO® Horizon Adventures: Bringing Terrain to Switch”. Unreal Fest Stockholm 2025. 2025. URL: <https://www.youtube.com/watch?v=heoXRhQv8cs>.
- [22] Unity Software Inc. *Cloth.enabled, Unity Documentation*. Available at <https://docs.unity3d.com/6000.3/Documentation/ScriptReference/Cloth-enabled.html>. Unity Software Inc. 2026.

- [23] John Elder. *Deep Learning With PyTorch*. <https://www.youtube.com/watch?v=kY14KfZQ1TI&list=PLCC340HNc0tpcgR9LEYSdi9r7XIbpbkpK1>. Accessed: 07-04-2026.
- [24] Epic Games, Inc. *Hardware and Software Specifications*. <https://dev.epicgames.com/documentation/en-us/unreal-engine/hardware-and-software-specifications-for-unreal-engine>. Accessed: 06-04-2026.
- [25] Epic Games, Inc. *Introduction to Performance Profiling and Configuration*. <https://dev.epicgames.com/documentation/en-us/unreal-engine/introduction-to-performance-profiling-and-configuration-in-unreal-engine>. Accessed: 03-04-2026.
- [26] Epic Games, Inc. *NNE - Neural Post Processing*. <https://dev.epicgames.com/community/learning/tutorials/7dr8/unreal-engine-nne-neural-post-processing>. Accessed: 12-05-2026.
- [27] Gamedev.js. *Gamedev.js Survey 2024 Report: State of Web Game Development in 2024*. <https://gamedevjs.com/survey/2024/>. Accessed: 06-04-2026.
- [28] Daniel Holden. *Machine Learning, Kolmogorov Complexity, and Squishy Bunnies*. <https://theorangeduck.com/page/machine-learning-kolmogorov-complexity-squishy-bunnies>. Published 27-08-2019. Accessed: 06-04-2026.
- [29] Hypesio. *Batman's cape simulation in Batman: Arkham Asylum*. <https://hypesio.fr/en/batmans-cape-simulation-in-batman-arkham-asylum/>. Accessed: 03-04-2026.
- [30] Intel Corporation. *Singular Value Decomposition: LAPACK Computational Routines*. <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2025-0/singular-value-decomposition-lapack-computation.html>. Accessed: 08-04-2026.
- [31] International Business Machines Corporation. *What is principal component analysis (PCA)?* <https://www.ibm.com/think/topics/principal-component-analysis>. Accessed: 06-04-2026.
- [32] Christian Kardach and Plamen Tamnev. *Machine Learning Cloth Simulation Overview*. <https://discussions.unity.com/t/making-of-time-ghost-cloth-deformation-with-unity-sentis/1537842?page=2>. Accessed: 03-04-2026.
- [33] Jacob Murel and Eda Kavlakoglu. *What is dimensionality reduction?* <https://www.ibm.com/think/topics/dimensionality-reduction>. Accessed: 03-04-2026.
- [34] Kostyantyn Oleschenko, Olexandr Makeev, and Nadiia Havrylenko. *How to get better fps on PC while porting games to consoles*. <https://www.gamedeveloper.com/programming/how-to-get-better-fps-on-pc-while-porting-games-to-consoles->. Accessed: 03-04-2026.
- [35] Robert Purchase. *Eidos expects Batman to score in the 90s*. <https://www.eurogamer.net/eidos-expects-batman-to-score-in-the-90s>. Eurogamer article. Accessed: 03-04-2026.

- [36] Magica Soft. *Magica Soft, Magica Cloth Documentation*. Available at https://magicasoft.jp/en/mc2_performance/. Magica Soft.
- [37] Suraj Subramanian et al. *Learn the Basics*. <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>. Accessed: 07-04-2026.
- [38] Unity Software Inc. *Inference Engine overview*. <https://docs.unity3d.com/Packages/com.unity.ai.inference@2.2/manual/index.html>. Accessed: 07-04-2026.
- [39] Video game Insights. *The Big Game Engine Report of 2025*. https://app.sensortower.com/vgi/assets/reports/The_Big_Game_Engines_Report_of_2025.pdf. Accessed: 06-04-2026.

A Appendix

A.1 Listings

```
def LoadArchive(archive):
    iarch = alembic.Abc.IArchive(BankPath + "0.abc")
    top = iarch.getTop()

    planeParentObject = top.children[0]
    clothMeshObject = planeParentObject.children[0]
    clothPolyMesh = alembic.AbcGeom.IPolyMesh(planeParentObject, "outputCloth1")

    clothSchemaReader = clothPolyMesh.getSchema()
    NumOfClothSamplesPerCache = clothSchemaReader.getNumSamples()

    clothSample = clothSchemaReader.getValue(alembic.Abc.ISampleSelector(0))
    positions = clothSample.getPositions()
    NumOfVertices = len(positions)
```

Listing A.1: Single cache loading abridged from
MultiAlembicFileTraining.py

```
NumWindowStartsPerCache = CacheSize / OverlapInterval
BatchStretch = 32 # How many batches in a row from same animation to train on
BatchSize *= BatchStretch # Process a sequence of batches from same cache
for Cache in range(0, NumCaches):
    for Offset in range(0, OffsetRange):
        WindowStartIndices = (np.arange(NumWindowStartsPerCache) * OverlapInterval)
            # Windows start at 0, 4, 8, etc...
        WindowStartIndices += Offset # Window starts shifted
        WindowStartIndices = WindowStartIndices[:len(WindowStartIndices)-int((
            WindowSize+Offset)/OverlapInterval)-1] # Make sure we have a multiple
            of large batches of windows
        BatchRemainder = len(WindowStartIndices) % BatchSize
        WindowStartIndices = WindowStartIndices[:len(WindowStartIndices)-
            BatchRemainder]

        # Batches of overlapping windows tagged with CacheID shuffled and put into
            one list of windows
        CacheArray = np.full((len(WindowStartIndices)), Cache)
        CacheAndWindowPairs = np.column_stack((CacheArray, WindowStartIndices))
        Batches = np.split(CacheAndWindowPairs, int(len(CacheAndWindowPairs)/
            BatchSize)) # Split into large batches
        WindowStartIndicesBatches += Batches # Add large batches to list of all
            large batches

    RandGen.shuffle(WindowStartIndicesBatches)
    AllWindowPairs = np.concatenate(WindowStartIndicesBatches, axis=0)

    TrainingWindowCutOffIndex = int(TrainingValidationRatio * len(AllWindowPairs))
    TrainingWindows = AllWindowPairs[0:TrainingWindowCutOffIndex]
    TestingWindows = AllWindowPairs[TrainingWindowCutOffIndex:]

return TrainingWindows, TestingWindows
```

Listing A.2: Window list generation with singular-cache batches for the
total-random training process

```
for y in range(0, height - 1):
    for x in range(0, width - 1):
        vert0 = (y*width) + x # bottom right
        vert1 = (y*width) + x + 1 # bottom left
        vert2 = ((y+1)*width) + x # top right
        vert3 = ((y+1)*width) + x + 1 # top left
        # Append faces clockwise
```

```

        faces.append([vert0, vert1, vert2])
        faces.append([vert1, vert3, vert2])
        ifaces_indices.extend([vert0, vert1, vert2, vert1, vert3, vert2])
iFaceCounts = setArray( alembic.Abc.Int32TPTraits, *((3,) * len(faces)) ) # Defines
        how faces are arranged
iFaces = setArray( alembic.Abc.Int32TPTraits, *(tuple(ifaces_indices)) )

```

Listing A.3: Defining the faces, and formatting of faces, indexed into the array of vertices.

```

clothSample = alembic.AbcGeom.OPolyMeshSchemaSample(TensorToPolymeshPoints(
    VertexPositions2), iFaces, iFaceCounts)
clothPolyMesh.set(clothSample)
xsamp.setTranslation(imath.V3d(BallPos[0].item(), 0.5, BallPos[1].item()))
BallXForm.set(xsamp)

```

Listing A.4: Writing samples to the schemas.

```

void ShapeClothFromPCA(float[] PCACoords)
{
    float[] ClothSample = new float[Vertices.Length * 3];
    // Add up relevant amount of each component based on PCACoords
    for (int comp = 0; comp < PCABase; comp++)
    {
        for (int i = 0; i < ClothSample.Length; i++)
        {
            ClothSample[i] += (PCACoords[comp] * PCAComponents[comp, i]);
        }
    }
    // Un-normalize and de-centre (add average cloth sample)
    for (int i = 0; i < ClothSample.Length; i++)
    {
        ClothSample[i] += PCAMean[i];
        ClothSample[i] *= ScaleFactor;
    }
    //Convert to vertices
    int j = 0;
    for (int v = 0; v < Vertices.Length; v++)
    {
        Vertices[v].x = ClothSample[j];
        Vertices[v].z = ClothSample[j + 1];
        Vertices[v].y = ClothSample[j + 2];
        j += 3;
    }
    ClothMesh.vertices = Vertices;
    ClothMesh.RecalculateNormals();
}

```

Listing A.5: PCA to mesh vertices.

```

var awaiter = outputTensor.ReadbackAndCloneAsync().GetAwaiter();
isInferencePending = true;
awaiter.OnCompleted(async () =>
...

```

Listing A.6: Asynchronous tensor callback.

```

[numthreads(64,1,1)]
void CSMain (uint3 id : SV_DispatchThreadID)
{
    uint vertex_index = id.x;
    if (vertex_index >= NumVertices)
    {
        return;
    }
    int vertex_start = (3 * vertex_index);

```

```

float3 deformed_position = float3(AverageVertexFrame[vertex_start],
    AverageVertexFrame[vertex_start + 1], AverageVertexFrame[vertex_start + 2])
;
// For this vertex, add up each part of it from each PCA component
for (uint i = 0; i < PCABasisSize; i+=1)
{
    float PCACoord = PCACurFrame[i];
    deformed_position.x += PCABasis[(i * AbsoluteVertexFrameSize) +
        vertex_start] * PCACoord;
    deformed_position.y += PCABasis[(i * AbsoluteVertexFrameSize) +
        vertex_start + 1] * PCACoord;
    deformed_position.z += PCABasis[(i * AbsoluteVertexFrameSize) +
        vertex_start + 2] * PCACoord;
}
// deformed_position *= VarianceFrame[i]; // If with variance
// Target Mesh approach from Unity Time Ghost
MeshBuffer.Store3(StrideValues.x * vertex_index + StrideValues.y, asuint(
    deformed_position));
}

```

Listing A.7: vectorised PCA deformation into mesh vertex buffer

```

PCABasisTranspose = new Vector3[(PCADData.Length / 3)];
for (int i = 0; i < numVerts; i++)
{
    for (int j = 0; j < PCABase; j++)
    {
        // PCABasis per each vertex
        PCABasisTranspose[j + (PCABase * i)] = new Vector3(
            PCADData[(j * VertexFrameSize) + (i * 3)],
            PCADData[(j * VertexFrameSize) + (i * 3) + 1],
            PCADData[(j * VertexFrameSize) + (i * 3) + 2]);
    }
}

```

Listing A.8: PCA deformation matrix transpose

```

transform.position = StartPos + new Vector3(Mathf.Sin(-(StartOffset + Time.time *
    SinSpeed2)) * 2, 0, Mathf.Sin(-(StartOffset / 2) + Time.time * SinSpeed1)) *
    7);
...
moveInputSmoothed = Vector2.Lerp(moveInputSmoothed, moveInput, Time.deltaTime * 5);
transform.position += new Vector3(moveInputSmoothed.x * speed * Time.deltaTime *
    -1, 0, moveInputSmoothed.y * speed * Time.deltaTime * -1);

```

Listing A.9: Ball automatic smooth movement.

```

var job = new CalculateClothSampleJob
{
    PCAMean = PCAMean_Native,
    PCACoords = PCACoords_Native,
    PCADData = PCADData_Native,
    PCABase = PCABase,
    VertexBufferStride = BufferStride,
    NumVertices = Vertices.Length,
    VertexBuffer = VertexBuffer
};
JobHandle dependency = default;
unsafe
{
    jobHandle = job.ScheduleParallelByRef(Vertices_Native.Length, 128, dependency);
}

```

Listing A.10: Issuing the job using established Native Buffers

A.2 Survey

Start Statement: This study aims to test the believability of different methods of cloth simulation for video games. You will see pairs of videos below that use different methods for simulating cloth in games. In each pair, simply select the video which you believe has a more believable looking cloth simulation (e.g. closest to how you imagine cloth would move based on the object colliding with it).

This form should take up to 5 minutes to complete.

- If you would like your response deleted before the end of the study please contact me
- You can ask for any further information about the purpose and the outcomes of the study.

There is optionally the space to enter your email for a voucher draw at the end of this form.

I give consent for my responses to be recorded and stored until the end of the study. I consent to the data gathered from this session being analysed for the purposes of this study. Data from this session will be anonymised in the report and any personal data collected for contact (e.g. voucher collection) will be destroyed on study completion in June 2026.

You understand the rights set out above: *I agree*

Task + Material Specification: The following videos are tests for different implementations of a heavy drape material in a video game. The game involves a ball rolling around the level. The ball will be able to interact with environmental props to make the world feel more responsive. They are testing out different implementations for the heavy drape material that will be found hanging in the world.

Per Pair X vx. Y

[Videos, Figure 4.6a]

From the above videos, please select the cloth which you think moves most believably (e.g. closest to how you imagine cloth would move based on the object colliding with it).

- Cloth Implementation X
- Cloth Implementation Y
- They are equally believable

How believable is implementation X?

- 1 - Unbelievable
- 2

- 3
- 4
- 5 - Believable

How believable is implementation Y?

- 1 - Unbelievable
- 2
- 3
- 4
- 5 - Believable

Why did you think the one you chose was more believable?

Voucher draw

Enter email for voucher enter: